

GEMM OPTIMIZATION WITH CUDA CORES



Meng Wang, Perkz Zheng
2021.07

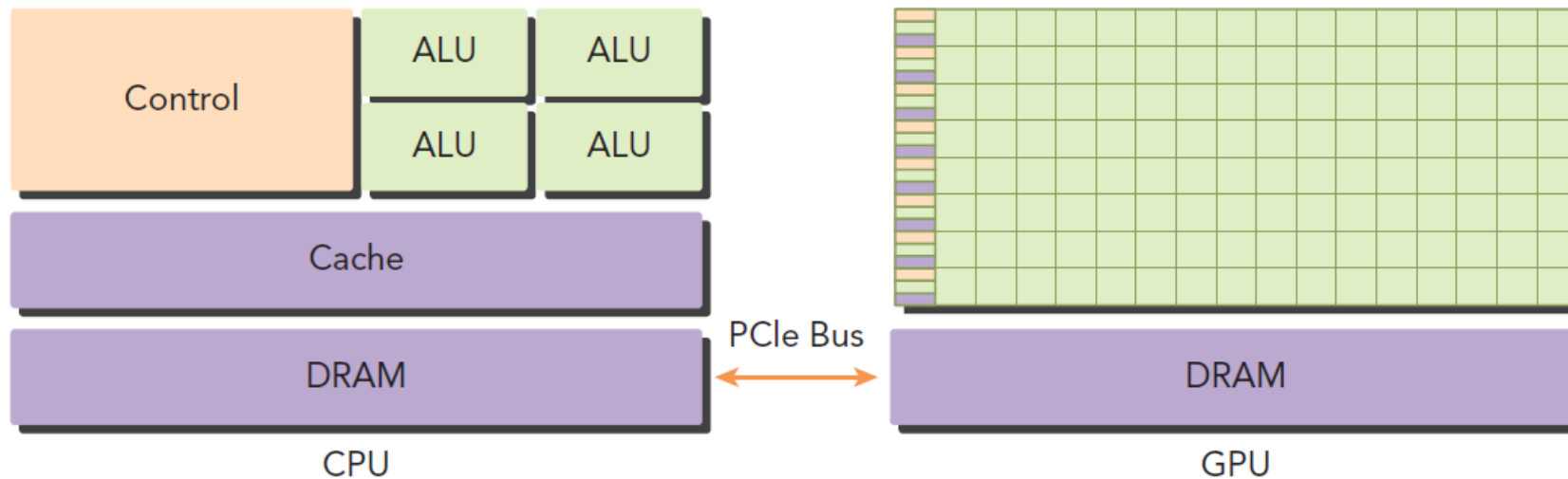
The background of the slide is a dark, almost black, field. It is populated with a network of thin, light green lines that crisscross the frame. At various points where these lines intersect or terminate, there are small, bright green circular dots. Some of these dots have a slight glow or halo effect. The overall impression is one of a complex, interconnected system, possibly representing a neural network or a data structure.

INTRODUCTION TO CUDA

GPU COMPUTING

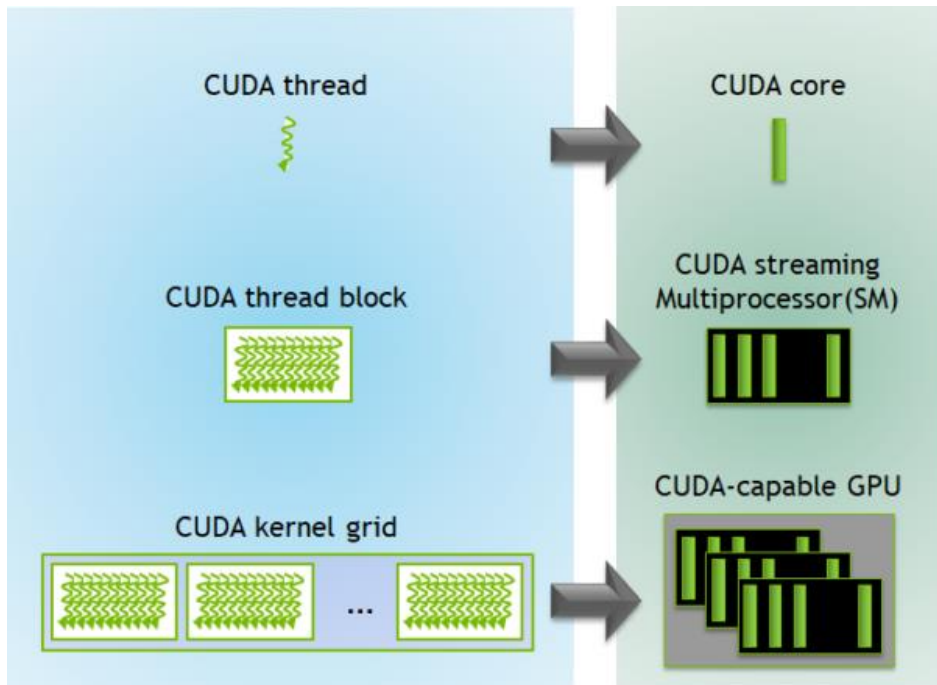
CORE CORES

- **CPU core**
 - relatively heavy-weight, designed for **complex control logic**, optimized for **sequential** programs
- **GPU core**
 - relatively light-weight, designed with simple control logic, optimized for **data-parallel** tasks, focusing on throughput of **parallel** programs

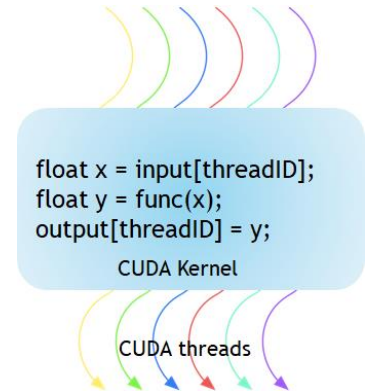


CUDA COMPUTE KERNEL HIERARCHY

Thread, Block, Grid

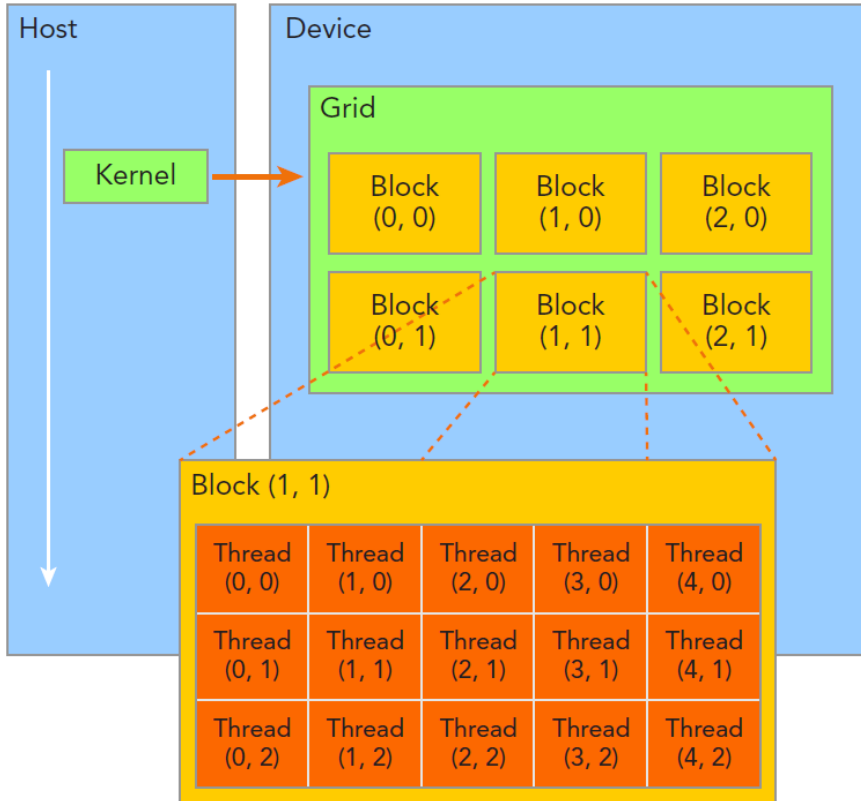


- **Thread**: sequential execution unit
 - All threads execute same sequential program
 - Threads execute in parallel
- **Block**: a group of threads
 - Executes on a single Streaming Multiprocessor (SM)
 - Threads within a block can cooperate
- **Grid**: a collection of thread blocks
 - Thread blocks of a grid execute across multiple SMs
 - Communication between blocks is expensive



CUDA COMPUTE KERNEL HIERARCHY

Thread, Block, Grid



- Execution configuration:
- `dim3 grid(3, 2, 1), block(5, 3, 1)`
- Built-in variables:
 - `threadIdx.[x y z]`
 - **thread index within a thread block**
 - `blockIdx.[x y z]`
 - **block index within the grid**
 - `blockDim.[x y z]`
 - **Number of threads in each block**
 - `gridDim.[x y z]`
 - **Number of blocks in the grid**

CUDA COMPUTE KERNEL HIERARCHY

Warps and Warp Scheduler

The multiprocessor creates, manages, schedules, and executes threads in groups of **32** parallel threads called *warps*.

- The threads in the same warp always executing the same instruction
- Instructions in the same warp execute serially
- Latency is caused by the dependency between the successive instructions in the same warp
- In the waiting time, other instructions from other warps can be executed
- Context switching is free
- A lot of warps can hide memory latency



CUDA COMPUTE KERNEL HIERARCHY

Warps and Warp Scheduler

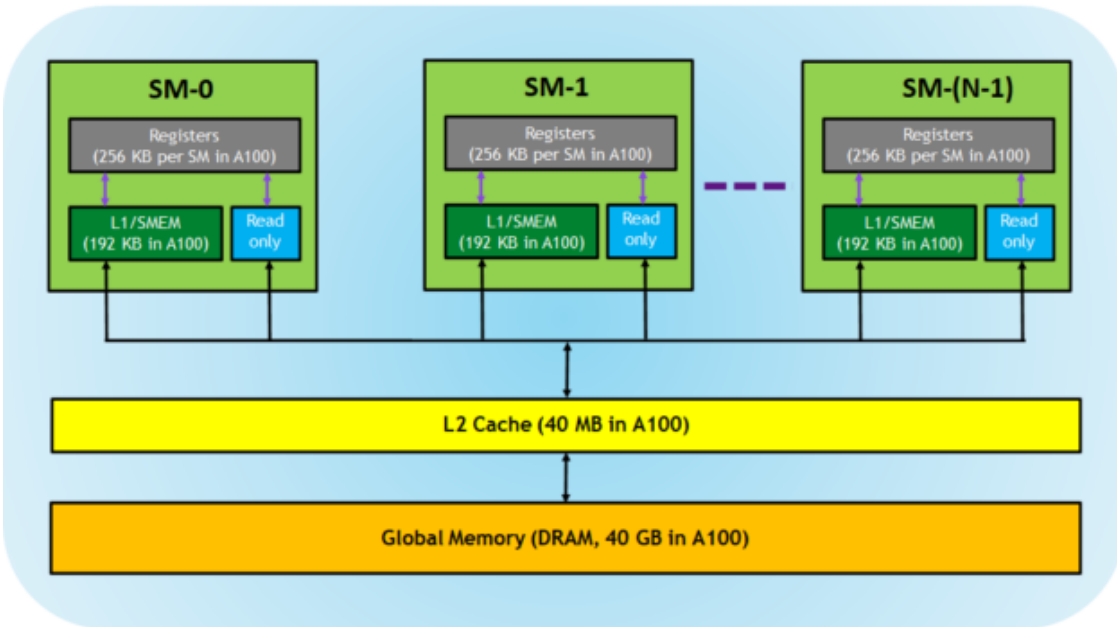
A warp is *stalled* when the warp is waiting

- an instruction fetch,
- a **memory** dependency (result of memory instruction),
- an **execution** dependency (result of previous instruction), or
- a **synchronization** barrier.



CUDA MEMORY HIERARCHY

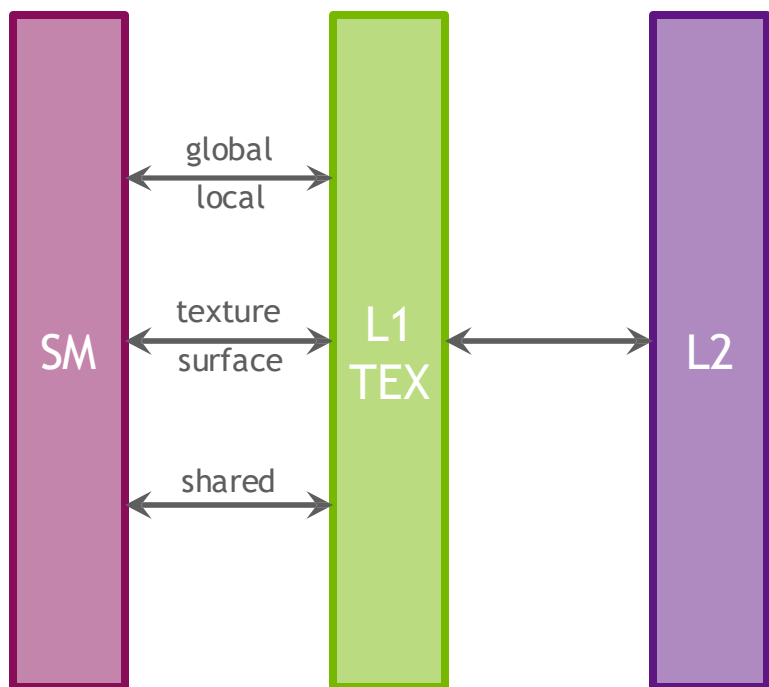
Global Memory, L2, L1/Shared, Read Only, RF



- **Registers**—These are private to each thread, which means that registers assigned to a thread are not visible to other threads. The compiler makes decisions about register utilization.
- **L1/Shared memory (SMEM)**—Every SM has a fast, on-chip scratchpad memory that can be used as L1 cache and shared memory. All threads in a CUDA block can share shared memory, and all CUDA blocks running on a given SM can share the physical memory resource provided by the SM.
- **Read-only memory**—Each SM has an instruction cache, constant memory, texture memory and RO cache, which is read-only to kernel code.
- **L2 cache**—The L2 cache is shared across all SMs, so every thread in every CUDA block can access this memory. The [NVIDIA A100 GPU](#) has increased the L2 cache size to 40 MB as compared to 6 MB in V100 GPUs.
- **Global memory**—This is the framebuffer size of the GPU and DRAM sitting in the GPU.

MENTAL MODEL: L1TEX CACHE

Cache Properties



Unified Cache / Shared Memory
192KB per SM, Variable split (~15MB Total for GPU)

Memory access granularity = 32 Bytes = 1 sector
Cache line is 128 Bytes, made of 4 sectors

Memory Reads

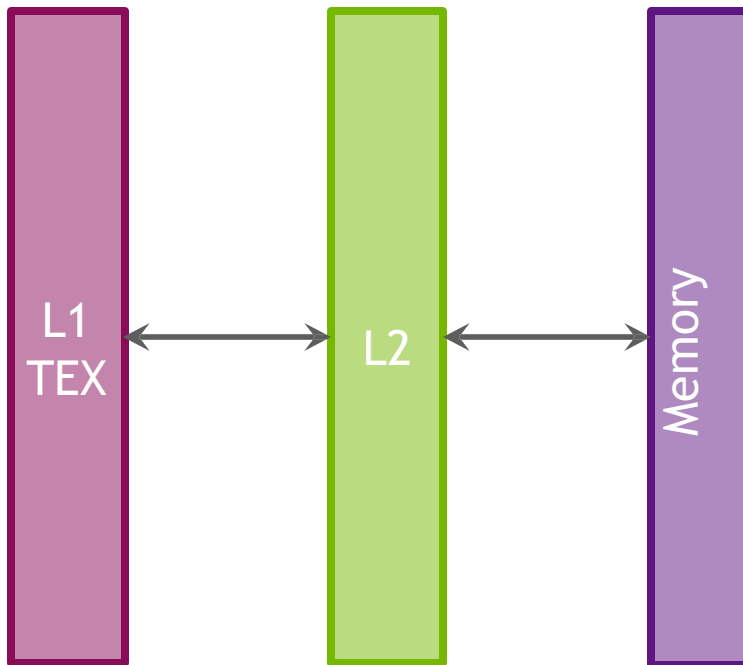
Unit of data moved: Full sector
Check if data is in L1TEX (if not, check L2)

Memory Writes

Unit of data moved: Partial sector
Cache policy is write-through (Write to L1 AND L2)
(can be influenced by NVCC flag or inline PTX)

MENTAL MODEL: L2 CACHE

Cache Properties



40 MB L2 Cache
2 partitions, 40 slices per partition
(1555 GB/s bandwidth, 5120 Bytes/clock read)

Memory access granularity = 32 Bytes = 1 sector
Cache line is 128 Bytes, made of 4 sectors

Memory Reads

Unit of data moved: Full Sector
Check if data is in L2 (if not, access from source memory)

Memory Writes

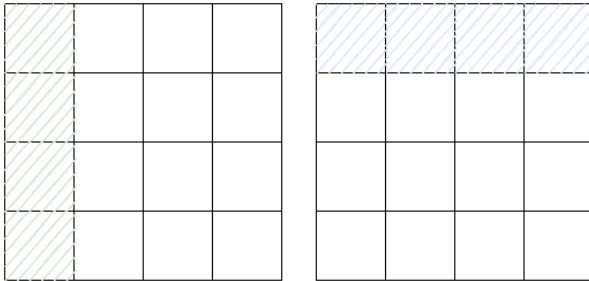
Unit of data moved: Partial Sector
Cache policy is write-back: Flushes data to Global Memory when needed
(can be influenced by CUaccessProperty)

<https://www.nvidia.com/en-us/on-demand/session/gtcspring21-s32089/>

CUaccessProperty: https://docs.nvidia.com/cuda/cuda-driver-api/group_CUDA_TYPES.html#group_CUDA_TYPES_1gbf4806140e86865b45fc144941a6ced5

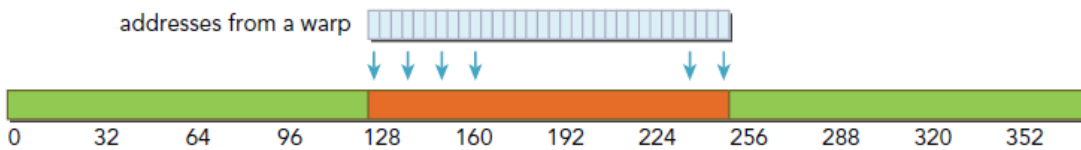
CUDA MEMORY ACCESS

Coalesced Global Memory Access



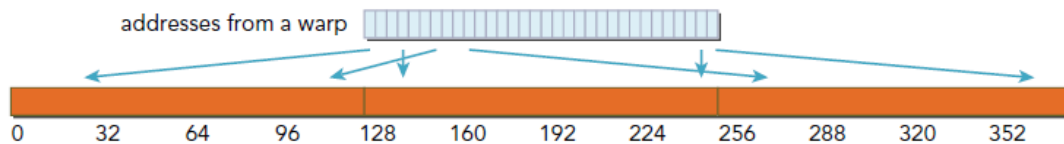
Coalesced memory accesses occur when all 32 threads in a warp access a contiguous chunk of memory. Use the least number of transactions to service the maximum number of memory requests.

Assume L1 cache is enabled, then cache line size is 128 byte.



1st case: 128 bytes / 128 bytes (actual / ideal)

2nd case: 384 bytes / 128 bytes (actual / ideal)



Worst case: 32N bytes / N bytes (actual / ideal)

CUDA MEMORY ACCESS

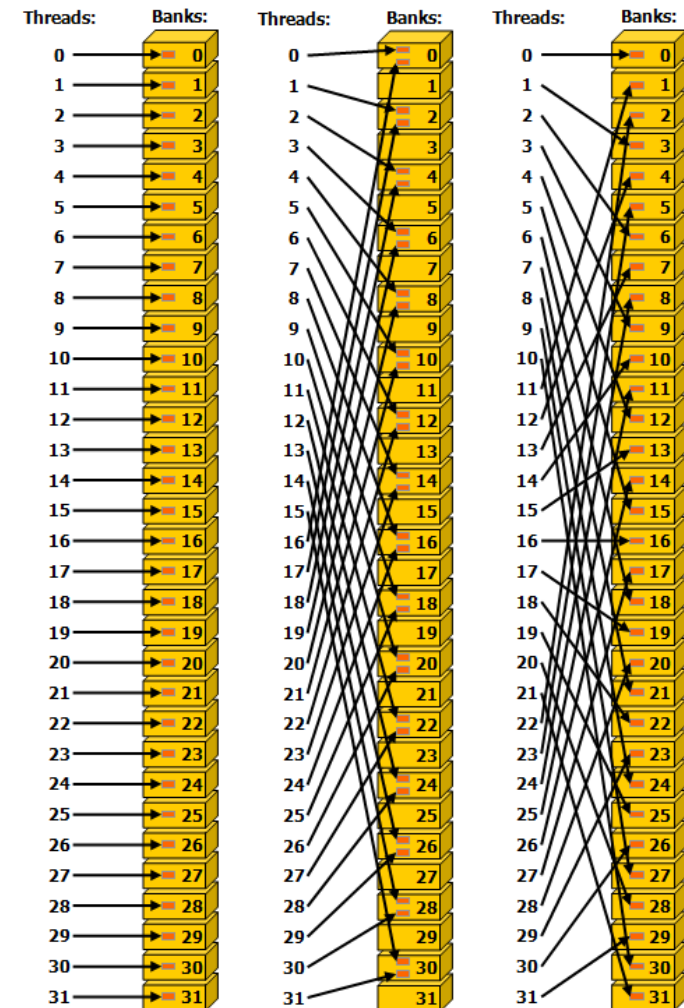
Shared Memory Bank Conflicts

When multiple addresses in a shared memory request fall into the same memory bank, a *bank conflict* occurs, causing the request to be replayed an instruction fetch.

There are 32 banks with successive 32-bit words.

- **Parallel access:** multiple addresses accessed across multiple banks
- **Serial access:** multiple addresses accessed within the same banks

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#shared-memory-5-x>



CUDA MEMORY ACCESS

Shared Memory Bank Conflicts

Memory padding is one way to avoid bank conflicts.

Bank 0 Bank 1 Bank 2 Bank 3 Bank 4 padding

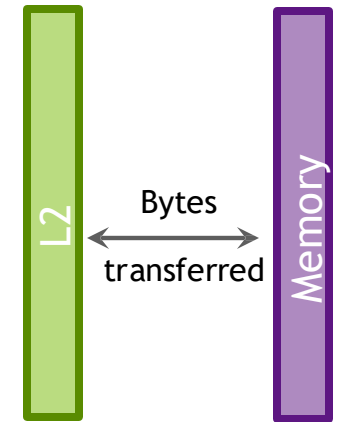
0	1	2	3	4	
0	1	2	3	4	
0	1	2	3	4	
0	1	2	3	4	
0	1	2	3	4	

Bank 0 Bank 1 Bank 2 Bank 3 Bank 4

0	1	2	3	4
	0	1	2	3
4		0	1	2
3	4		0	1
2	3	4		0
1	2	3	4	

NSIGHT COMPUTE

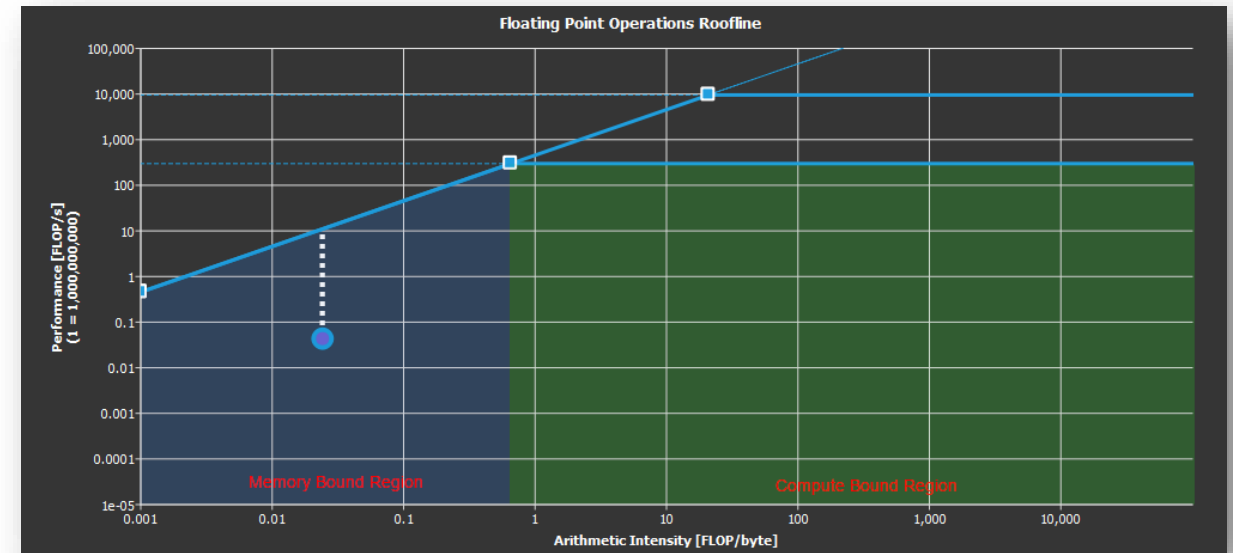
Roofline and Arithmetic Intensity



Arithmetic Intensity (FLOP/Byte)

$$AI \left[\frac{\text{FLOP}}{\text{byte}} \right] = \frac{N_{FLOPS}}{\text{bytes transferred}}$$

- NUM_FLOPS: $M \times N \times K \times 2$
- Bytes transferred: Load/Store from Global Memory to L2
- Measure of data locality (**data reuse**)
- Can be very different from total loads/stores (bytes requested) due to cache reuse



<https://developer.download.nvidia.com/video/gputechconf/gtc/2019/presentation/s9624-performance-analysis-of-gpu-accelerated-applications-using-the-roofline-model.pdf>

<https://developer.nvidia.com/blog/accelerating-hpc-applications-with-nsight-compute-roofline-analysis/>



GEMM OPTIMIZATION STAGE 1

GENERAL MATRIX PRODUCT

Basic Definition

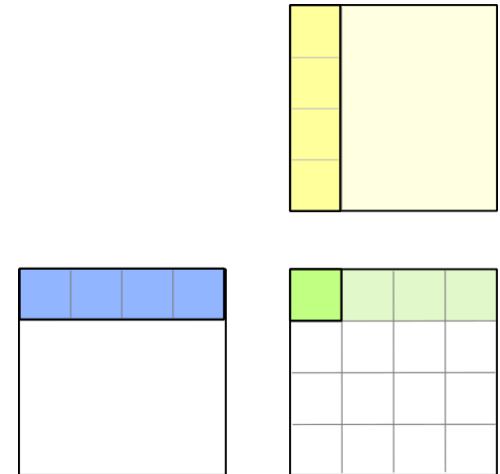
General matrix product

$$C = \alpha \text{ op}(A) * \text{ op}(B) + \beta C$$

C is M -by- N , $\text{op}(A)$ is M -by- K , $\text{op}(B)$ is K -by- N

Compute independent dot products

```
// Independent dot products
for (int i = 0; i < M; ++i)
    for (int j = 0; j < N; ++j)
        for (int k = 0; k < K; ++k)
            C[i][j] += A[i][k] * B[k][j];
```



GENERAL MATRIX PRODUCT

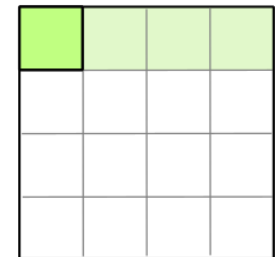
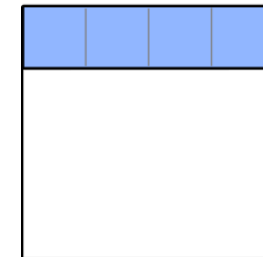
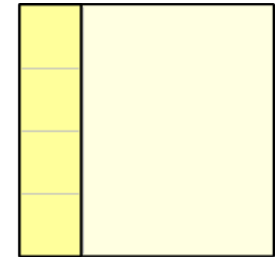
Experiment case

General matrix product

$$C = \alpha \text{ op}(A) * \text{ op}(B) + \beta C$$

C is M -by- N , $\text{op}(A)$ is M -by- K , $\text{op}(B)$ is K -by- N

- $M = N = 4096$, $K = 2048$ ($\alpha = 1$, $\beta = 0$, M , N , K are favorable integer multiples of tile size)
- **CUDA Version:** CUDA 11.1 under 20.12-tf1-py3 image
- **GPU:** A100-PCIE-40G on BFS
- **Warp Up:** 100 iterations
- **Compute:** 100 iterations



GEMM OPTIMIZATION STAGE 1

1. Base Line

Base Line:

Each thread corresponds to one element in the matrix.

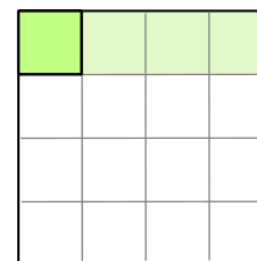
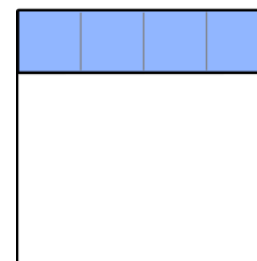
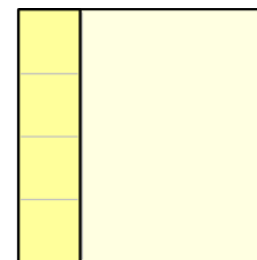
Assume block size (32, 32), grid size (128, 128)

Performance= 1861.22 GFlop/s, Time= 36.922 msec

How to improve ?

Memory Dependency: warps stall because of frequent global memory access

Elements can be reused by threads → **shared memory !**



GEMM OPTIMIZATION STAGE 1

1. Base Line

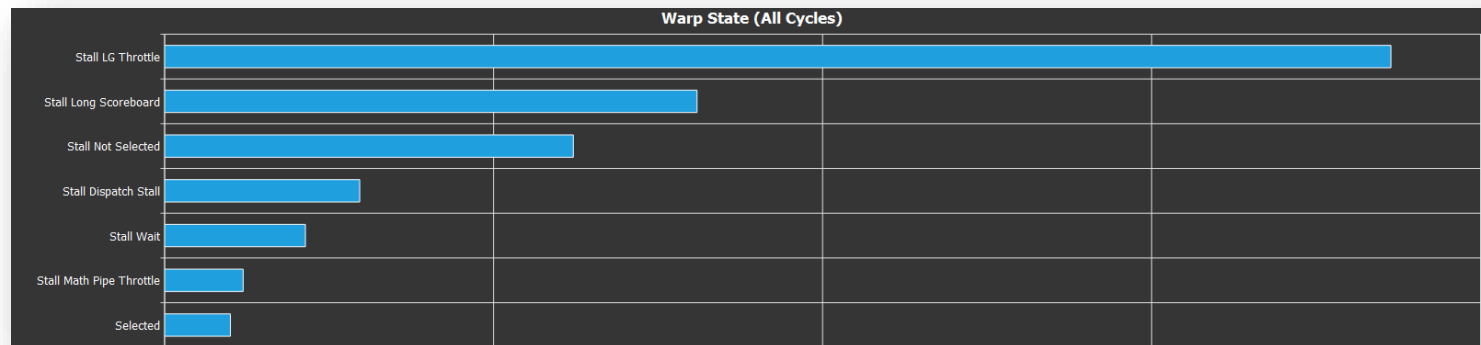
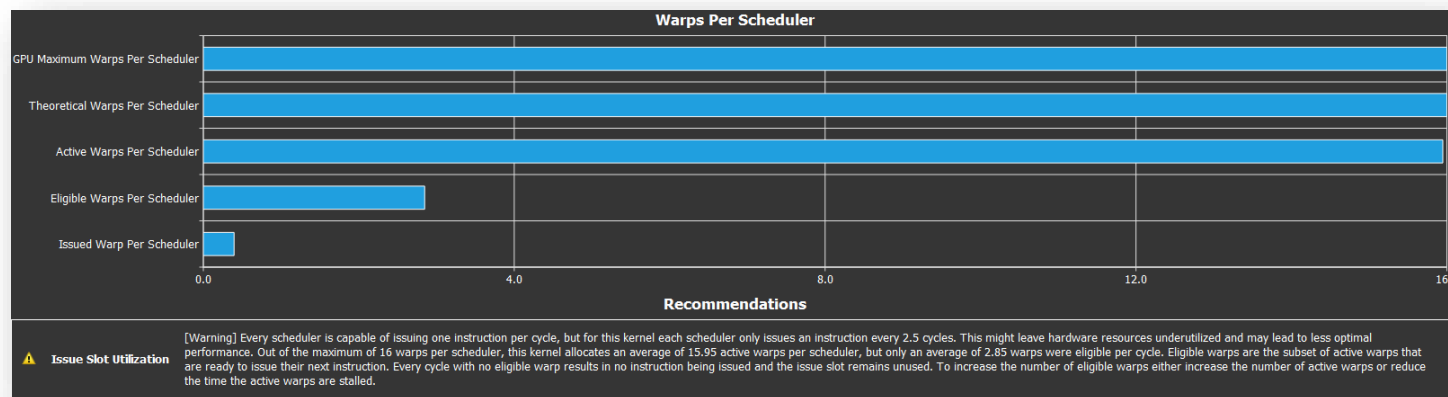
Base Line:

How to improve ?

Memory Dependency:

warps stall because extremely frequent memory instructions, according to the guided analysis rule

Elements can be reused by threads → **shared memory !**



GEMM OPTIMIZATION STAGE 1

2. Shared Memory and Block Tile

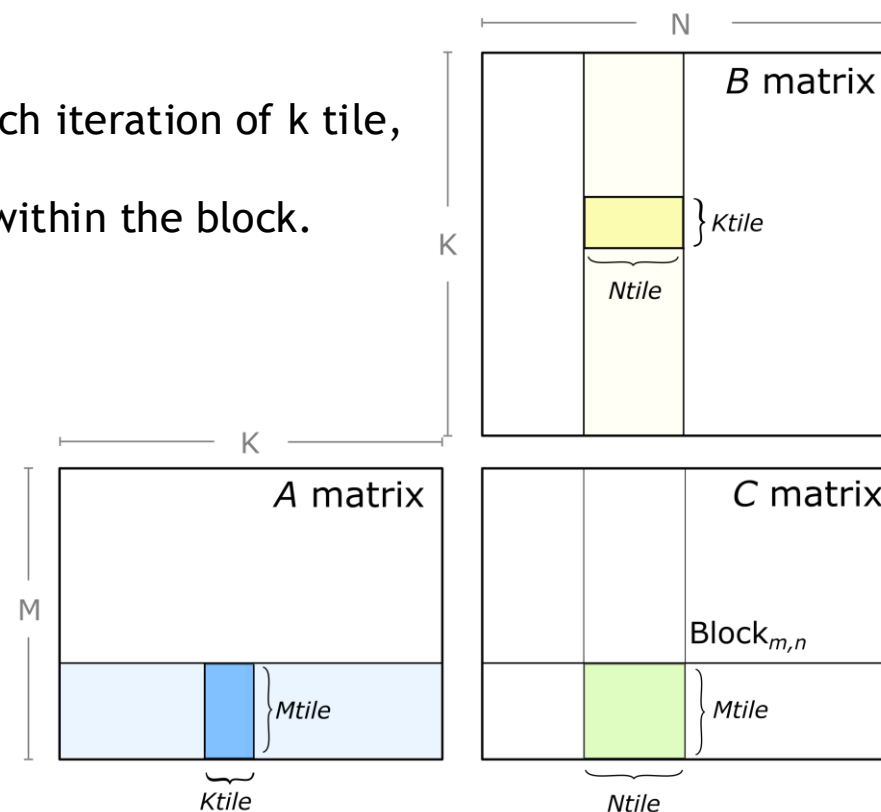
Shared Memory and Block Tile:

Each block stores needed A tile, B tile into shared memory for reuse. In each iteration of k tile,

The process is load from global memory to shared memory, inner product within the block.

Assume block size (32, 32), tile size (32, 32), grid size (128, 128)

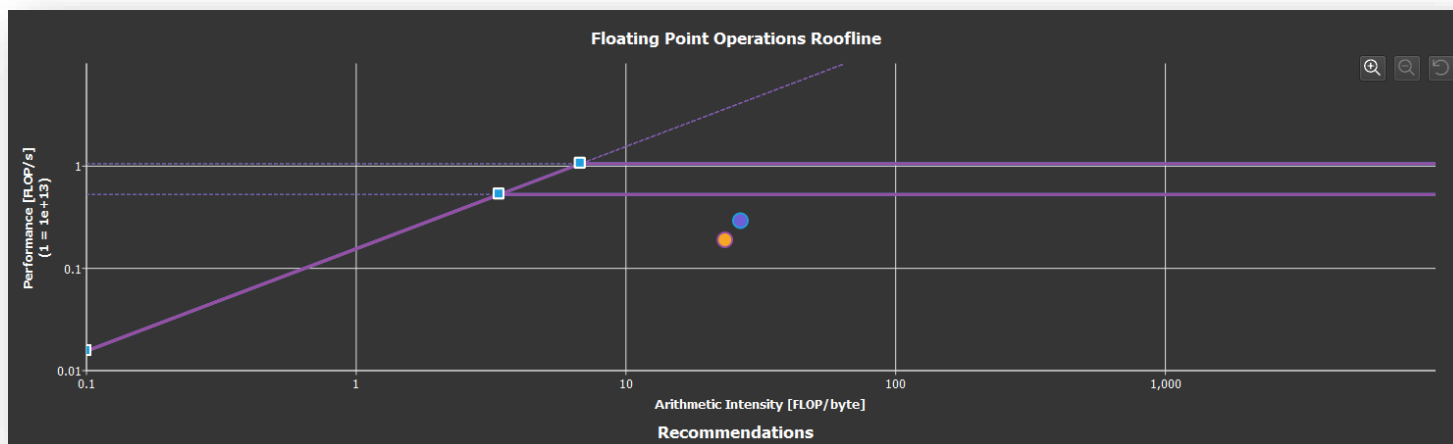
Performance= 2862.98 GFlop/s, Time= 24.003 msec



GEMM OPTIMIZATION STAGE 1

2. Shared Memory and Block Tile

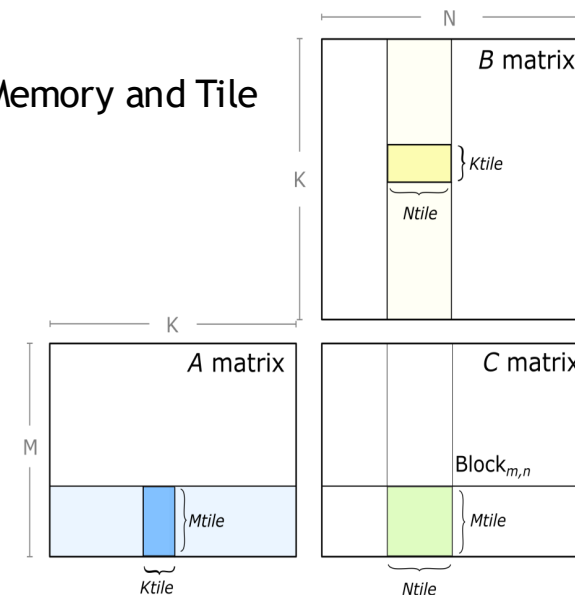
Shared Memory and Tile:



Baseline



Shared Memory and Tile



How to improve ?

GEMM OPTIMIZATION STAGE 1

2. Shared Memory and Block Tile

```
template <int BLOCKDIM> __global__ void MatrixMulCUDA(float *C, float *A,
float *B, int M, int N, int K) {

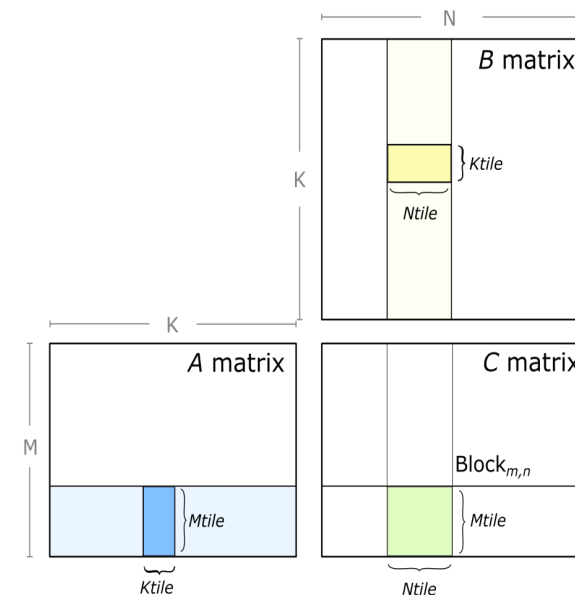
    __shared__ float As[BLOCKDIM][BLOCKDIM];
    __shared__ float Bs[BLOCKDIM][BLOCKDIM];

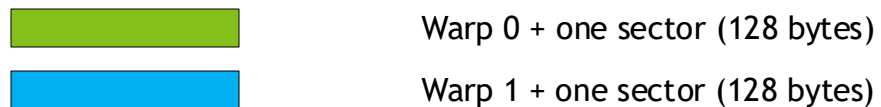
    uint idx = threadIdx.x + blockIdx.x * blockDim.x;
    uint idy = threadIdx.y + blockIdx.y * blockDim.y;

    float sum = 0.f;

    #pragma unroll
    for (size_t k = 0; k < K; k += BLOCKDIM)
    {
        As[threadIdx.y][threadIdx.x] = A[idy*K + k + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(k + threadIdx.y)*N + idx];
        __syncthreads();

        #pragma unroll
        for (size_t j = 0; j < BLOCKDIM; j++)
        {
            sum += As[threadIdx.y][j]*Bs[j][threadIdx.x];
        }
        __syncthreads();
    }
    C[idy*N + idx] = sum;
}
```

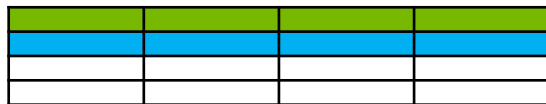




Warp 0 + one sector (128 bytes)

Warp 1 + one sector (128 bytes)

A Load from Global to Shared (32,32)



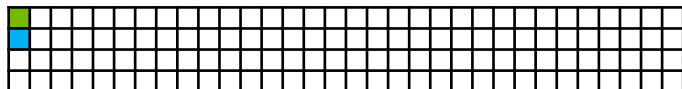
•
•
•
•

28 more rows

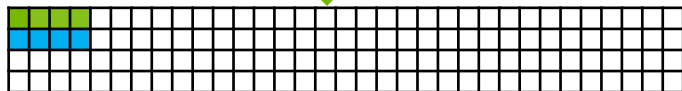
LDG 32 (1)
STS 32 (1)

STS 32 (1)

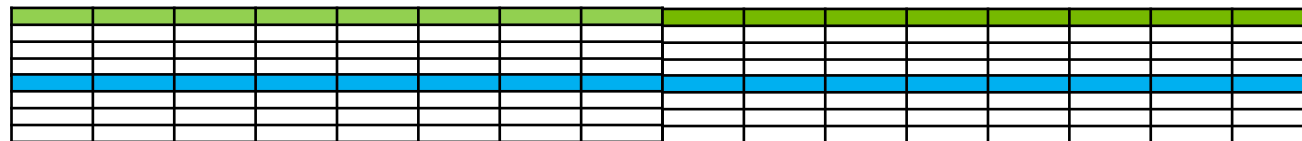
Compute: A load from Shared (1 * 1 * 32)



LDS 128 (8)

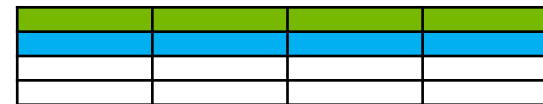


STG 32 (1)



2. Shared Memory and Block Tile

LDG 32 (1)
STS 32 (1)



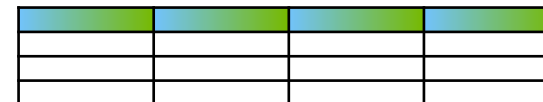
•
•
•
•

28 more rows

LDG 32 (1)
STS 32 (1)

STS 32 (1)

Compute: B load from Shared ($1 * 1 * 32$)



Compute: A load from Shared ($1 * 1 * 32$)

Memory Access: No Problem

GEMM OPTIMIZATION STAGE 1

2. Shared Memory and Block Tile

Shared Memory
and Block Tile:

Unrolling Optimization:

4 LDS.32 = LDS.128

One instruction, one
request

109	template <int BLOCKDIM> __global__ void MatrixMulCUDA(float *C, float *A,	6	524,288	16,777,216					
110	float *B, int M, int N, int K) {	0							
111		0							
112	__shared__ float As[BLOCKDIM][BLOCKDIM];	0							
113	__shared__ float Bs[BLOCKDIM][BLOCKDIM];	0							
114		0							
115	uint idx = threadIdx.x + blockIdx.x * blockDim.x;	342	4,718,592	150,994,944					
116	uint idy = threadIdx.y + blockIdx.y * blockDim.y;	286	3,670,016	117,440,512					
117		0							
118	float sum = 0;	0							
119		0							
120	#pragma unroll	0							
121	for (size_t k = 0; k < K; k += BLOCKDIM)	13,228	170,393,600	5,419,040,768					
122	{	0							
123	As[threadIdx.y][threadIdx.x] = A[idy*K + k + threadIdx.x];	64,218	203,948,032	6,526,337,024	Global, Shared	Load, Store	32(2)	33,554,432	
124	Bs[threadIdx.y][threadIdx.x] = B[(k + threadIdx.y)*N + idx];	7,184	403,701,760	12,918,456,320	Global, Shared	Load, Store	32(2)	33,554,432	
125	__syncthreads();	930	33,554,432	1,073,741,824					
126		0							
127	#pragma unroll	0							
128	for (size_t j = 0; j < BLOCKDIM; j++)	0							
129	{	0							
130	sum += As[threadIdx.y][j]*Bs[j][threadIdx.x];	182,402	2,415,919,104	77,309,411,328	Shared(40)	Load(40)	32(32), 128(8)		
131	}	0							
132	__syncthreads();	3,716	33,554,432	1,073,741,824					
133	}	0							
134	C[idy*N + idx] = sum;	66	2,097,152	67,108,864	Global	Store	32	524,288	
135	}	89	524,288	16,777,216					
136		0							

GEMM OPTIMIZATION STAGE 1

2. Shared Memory and Block Tile

Shared Memory and Block Tile:

How to improve (Stall MIO Throttle)?

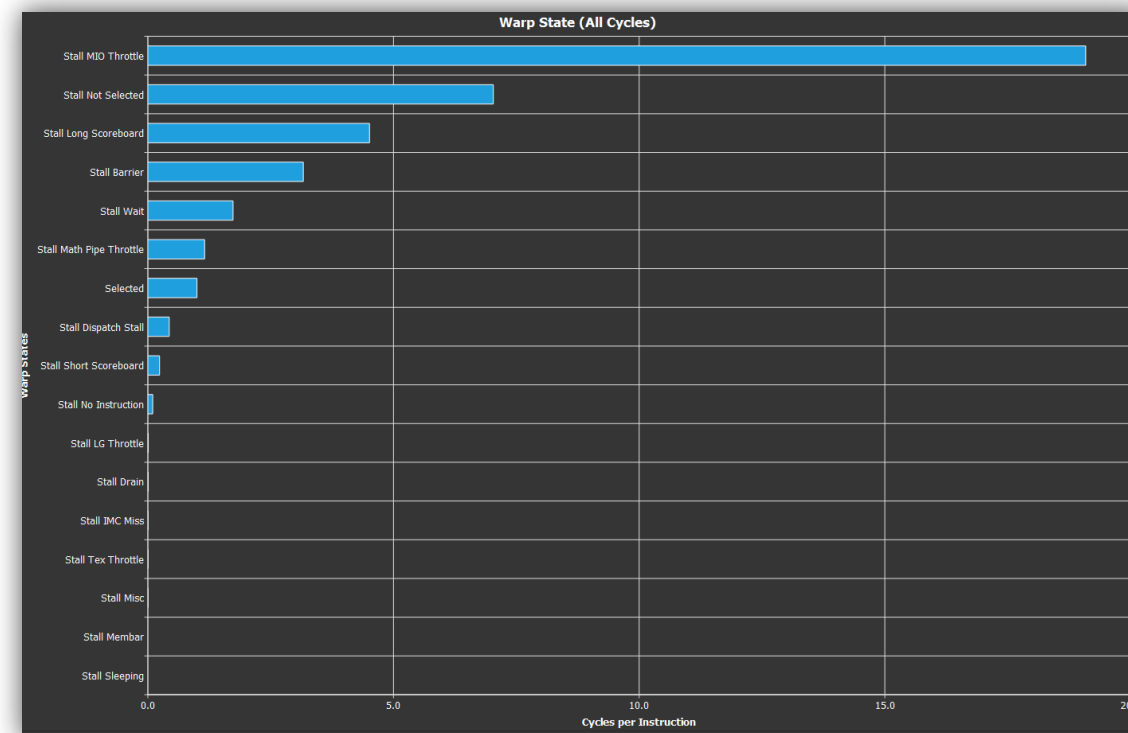
Reduce number of shared memory instructions

Compute / Memory higher $\rightarrow$ we can make block (32,32)

Deals with (128, 32) A tile, (32, 128) B tile. Each thread

Will compute (4, 4) $\rightarrow$

4x memory load / store, 16x FMA in one block



CPI Stall 'MIO Throttle'

[Warning] On average each warp of this kernel spends 19.1 cycles being stalled waiting for the MIO instruction queue to be not full. This represents about 49.6% of the total average of 38.5 cycles between issuing two instructions. This stall reason is high in cases of extreme utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions.

GEMM OPTIMIZATION STAGE 1

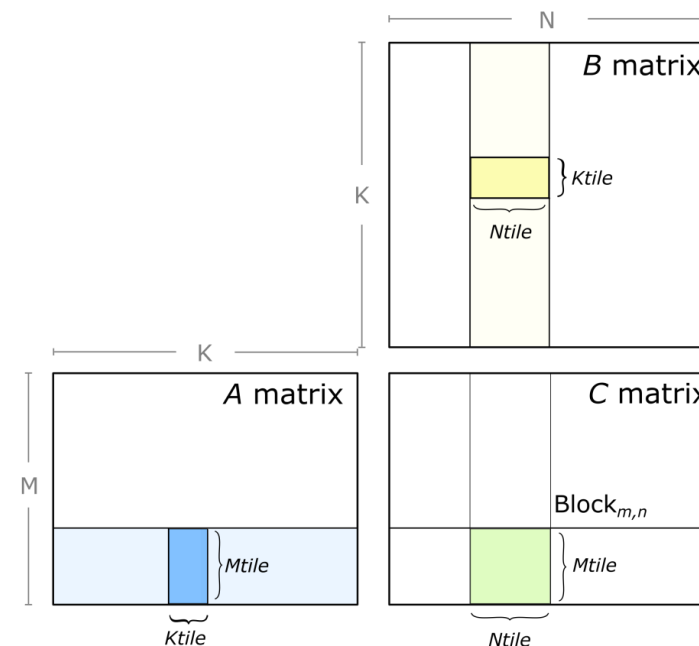
3. Thread Tile

Thread Tile:

Block (32, 32) loads A tile (128, 32), B tile (32, 128) and compute on (128, 128) tile

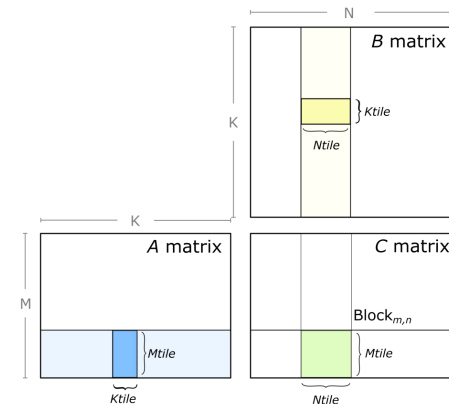
Assume block size (32, 32), grid size (32, 32)

Performance= 7989.69 GFlop/s, Time= 8.601 msec



GEMM OPTIMIZATION STAGE 1

3. Thread Tile: Process Summary



```
#pragma unroll
for (size_t k = 0; k < K; k += KTILE)
{
    loadAnB(ptrAs_store, ptrBs_store, ptrA, ptrB, K);

    ptrA += aStep;
    ptrB += bStep;
    __syncthreads();

    calcInnerBlockByInnerProduct(Csub, ptrAs_load, ptrBs_load);

    __syncthreads();
}

float * ptrC = C + (blockRow * mBlock + 4 * row) * N + blockCol * nBlock + 4 * col;

#pragma unroll
for (int i = 0; i < elemPerTy; i++) {
    *(float4 *) (ptrC + i * N) = *(float4 *) & Csub[i][0];
}
```

For each MK and NK tile

Load MK tile and NK tile
to Shared Memory

Inner Product Calculation
and Accumulation

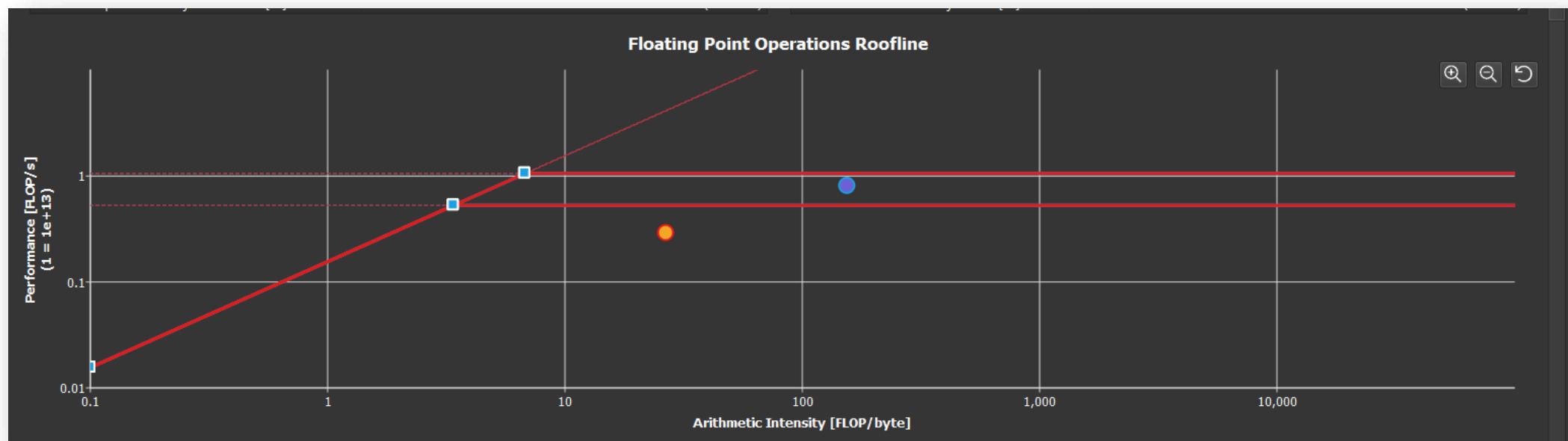
Store Result to Global Memory

GEMM OPTIMIZATION STAGE 1

3. Thread Tile

Thread Tile:

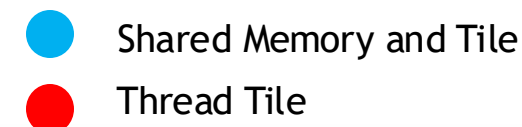
- Shared Memory and Tile
- Thread Tile



<https://developer.nvidia.com/blog/accelerating-hpc-applications-with-nsight-compute-roofline-analysis/>

GEMM OPTIMIZATION STAGE 1

3. Thread Tile



Instructions are issued in-order

If an instruction is not eligible, it stalls the warp

An instruction is eligible for issue if both are true:

- A **pipeline is available** for execution

Some pipelines need multiple cycles to issue a warp

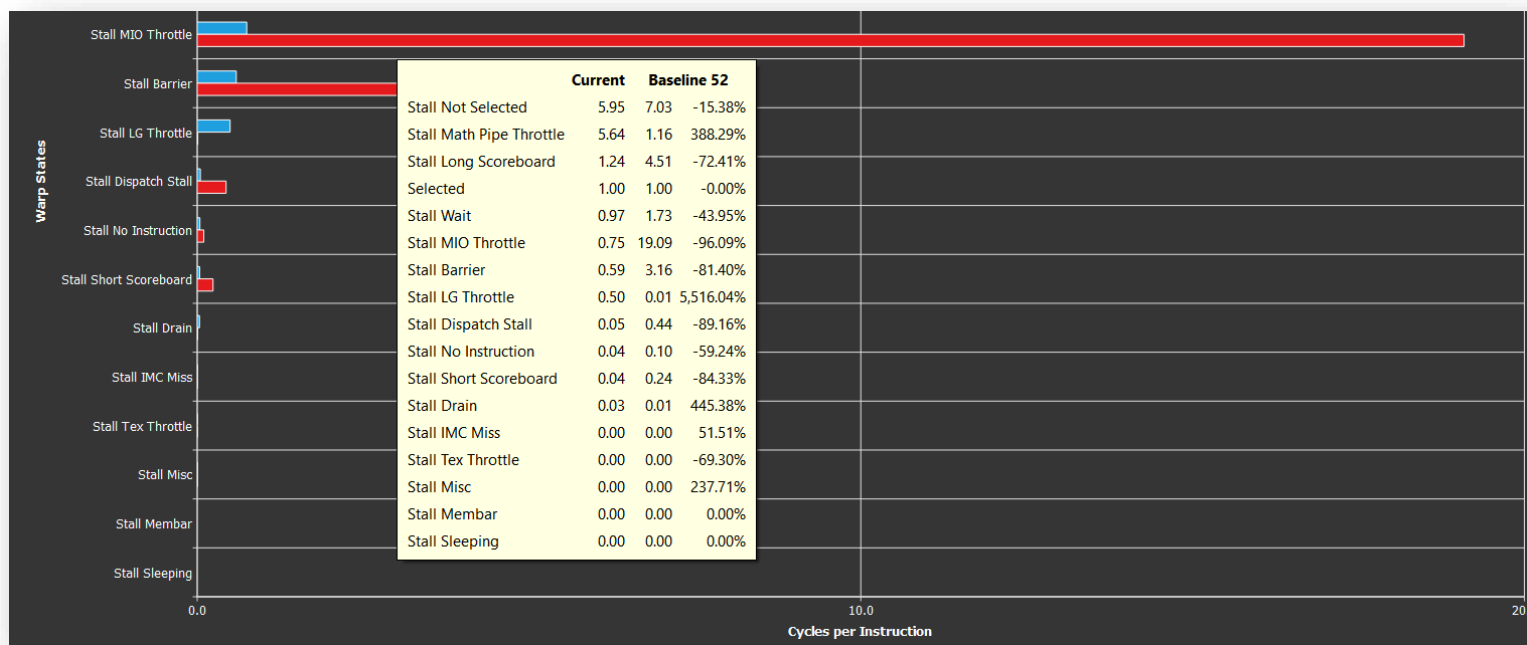
- All the **arguments are ready**

Argument isn't ready if a previous instruction

hasn't yet produced it

MIO Throttle:

Warp was stalled waiting for the MIO (memory input/output) instruction queue to be not full. This stall reason is high in cases of extreme utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions.



GEMM OPTIMIZATION STAGE 1

3. Thread Tile



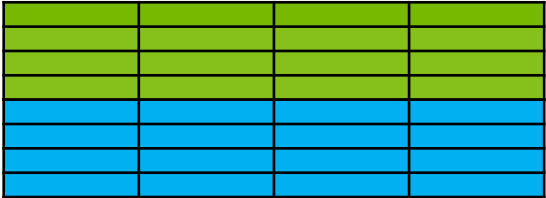
Warp 0 + one sector (128 bytes)



Warp 1 + one sector (128 bytes)

A Load from Global to Shared (128,32)

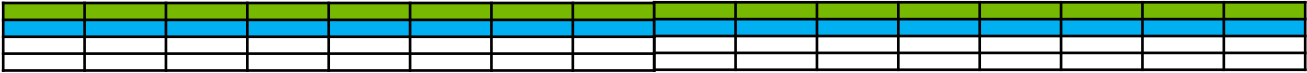
LDG 32 (4)
STS 32 (4)



120 more rows

B Load from Global to Shared (32, 128)

LDG 32 (4)
STS 32 (4)



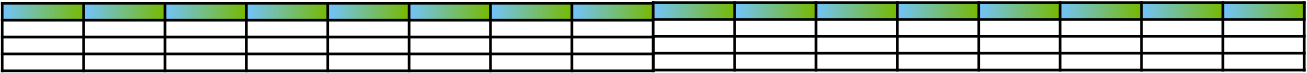
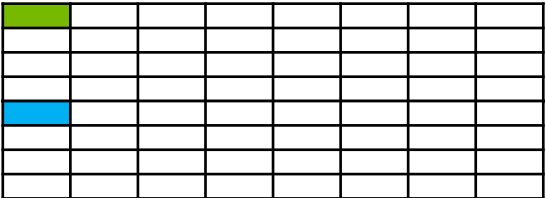
28 more rows

Compute: A load from Shared (4 * 4 * 32)

Compute: B load from Shared (4 * 4 * 32)

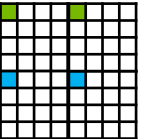
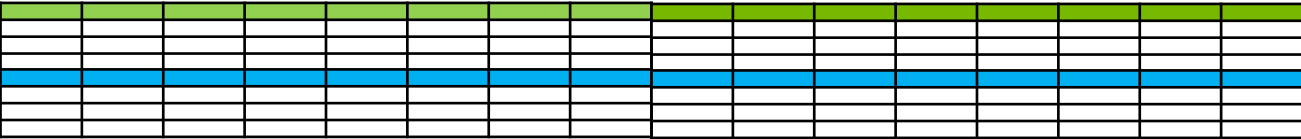
LDS 128 (32)

LDS 128 (32)



Compute: A load from Shared (4 * 4 * 32)

STG 32 (16)



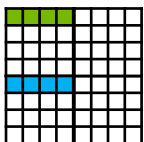
Uncoalesced global memory store

GEMM OPTIMIZATION STAGE 1

3. Thread Tile

Unrolling optimizes 4 LDS.32 to LDS.128

88 __inline__ __device__ void calcInnerBlockByInnerProduct(float Csub[elemPerTy][elemPerTx], f	0					
89 {	0					
90 #pragma unroll	0					
91 for (int i = 0; i < elemPerTy; i++)	0					
92 {	0					
93 #pragma unroll	0					
94 for (int j = 0; j < elemPerTx; j++)	0					
95 {	0					
96 #pragma unroll	0					
97 for (int k = 0; k < kBlock; k++)	0					
98 {	0					
99 Csub[i][j] += (ptrAs + i * kBlock + k)[0] * (ptrBs + k * nBlock + j)[0];	63,109	1,208,057,856	38,657,851,392	Shared(64)	Load(64)	128(64)
100 }	0					
101 }	0					
102 }	0					
103 }	0					
104	0					
105 /**	0					



GEMM OPTIMIZATION STAGE 1

3. Thread Tile

	Shared Memory				
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	134,217,728 (-90.00%)	134,217,728 (-90.00%)	402,653,184 (-75.02%)	56.65 (-30.36%)	0 (-100.00%)
Shared Load Matrix					
Shared Store	16,777,216 (-75.00%)	16,777,216 (-75.00%)	16,777,216 (-75.00%)	0.59 (-30.31%)	1,534,552 (-77.21%)

Shared Memory and Block Tile:

LDS 128 (8) + LDS 32 (32) → number of instructions = $128 \times 128 \times 64 \times 32 \times (8 + 32) = 1,342,177,280$

Number of Wavefronts = $128 \times 128 \times 64 \times 32 \times (8 \times 2 + 32 \times 1) = 1,611,766,371$

Thread Tile:

LDS 128 (32) + **LDS 128 (32)** → number of instructions = $32 \times 32 \times 64 \times 32 \times (32 + 32) = 134,217,718$ (- 90%)

Number of Wavefronts = $32 \times 32 \times 64 \times 32 \times (32 \times 2 + 32 \times 4) = 402,653,184$ (- 75 %)

Num of Instructions = $\text{blockDim.x} \times \text{blockDim.y} \times \text{Num_Kblocks} \times \text{Num_warps_per_block} \times \text{Num_Instruction_per_warp}$

GEMM OPTIMIZATION STAGE 1

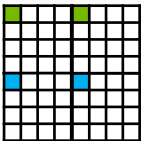
3. Thread Tile

STG is not coalesced, 4 STG.32

How to Optimize ?

- Each thread compute elements with a stride of 32 float (LDS.32, STG.32 with stride of 32) → coalesced global memory store → unoptimized LDS.32
- Cast to float4 when storing

160		0	0				
161	#pragma unroll	0	0				
162	for (int i = 0; i < elemPerTy; i++) {	0	0				
163	#pragma unroll	0	0				
164	for (int j = 0; j < elemPerTx; j++) {	0	0				
165	*(ptrC + i * N + j) = Csub[i][j];	3,178	3,132	720,896	23,068,672	Global(16)	Store(16) 32(16)
166	}	0	0				
167	}	0	0				
168		0	0				
169	}	344	343	32,768	1,048,576		
170		0	0				

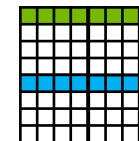


GEMM OPTIMIZATION STAGE 1

3. Thread Tile

STG is coalesced, STG.128

158		
159	<code>float * ptrC = C + (blockRow * mBlock + 4 * row) * N + blockCol * nBlock + 4 * col;</code>	
160		
161	<code>#pragma unroll</code>	
162	<code>for (int i = 0; i < elemPerTy; i++) {</code>	
163	<code> // #pragma unroll</code>	
164	<code> // for (int j = 0; j < elemPerTx; j++) {</code>	
165	<code> // *(ptrC + i * N + j) = Csub[i][j];</code>	
166	<code> // }</code>	
167	<code>//float4 stg</code>	
168	<code>*(float4 *) (ptrC + i * N) = *(float4 *) &Csub[i][0];</code>	Global(4) Store(4) 128(4)
169	<code>}</code>	
170		
171	<code>}</code>	



L1/TEX Cache									
	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	
Local Load	0 (+0.00%)	0 (+0.00%)		0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	
Global Load	16,777,216 (+0.00%)	16,777,216 (+0.00%)		2.39	67,108,864 (+0.00%)	4 (+0.00%)	0 (+0.00%)	2,147,483,648 (+0.00%)	
Global Load To Shared Store (access)	0 (+0.00%)	0 (+0.00%)		(+1.46%)	0 (+0.00%)	0 (+0.00%)			
Global Load To Shared Store (bypass)	0 (+0.00%)	0 (+0.00%)			0 (+0.00%)	0 (+0.00%)	-	0 (+0.00%)	
Surface Load	0 (+0.00%)	0 (+0.00%)		0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	
Texture Load	0 (+0.00%)	0 (+0.00%)		0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	
Global Store	131,072 (-75.00%)	131,072 (-75.00%)		0.02 (-73.61%)	2,097,152 (-75.00%)	16 (+0.00%)	0 (-100.00%)	67,108,864 (-75.00%)	2,097,152 (-75.00%)

GEMM OPTIMIZATION STAGE 1

3.1. Thread Tile (8, 4)

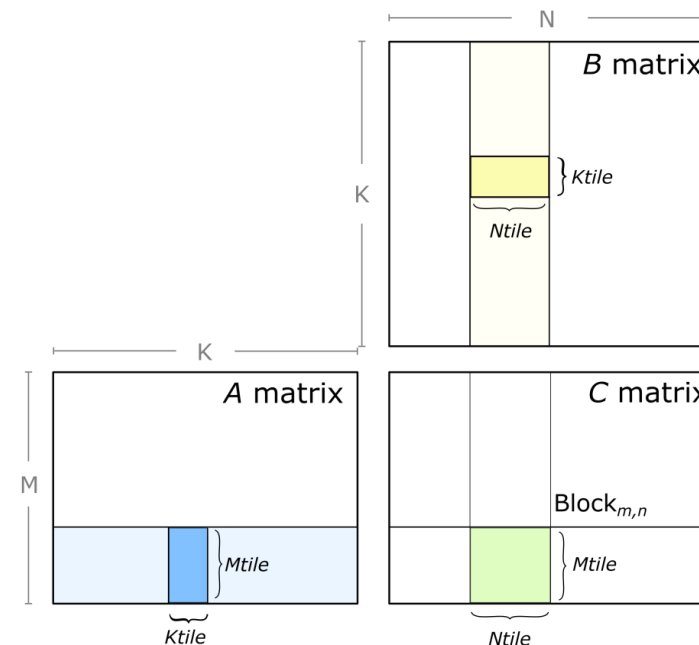
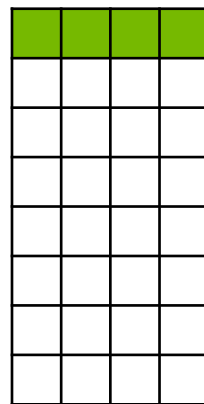
Thread Tile:

Block (32, 16) loads A tile (128, 16), B tile (16, 128) and compute on (128, 128) tile

Assume block size (32, 16), grid size (32, 32)

Each thread will compute (8, 4) C-sub-matrix.

Performance= 8431.95 GFlop/s, Time= 8.15 msec



GEMM OPTIMIZATION STAGE 1

3.2. Thread Tile (16, 4)

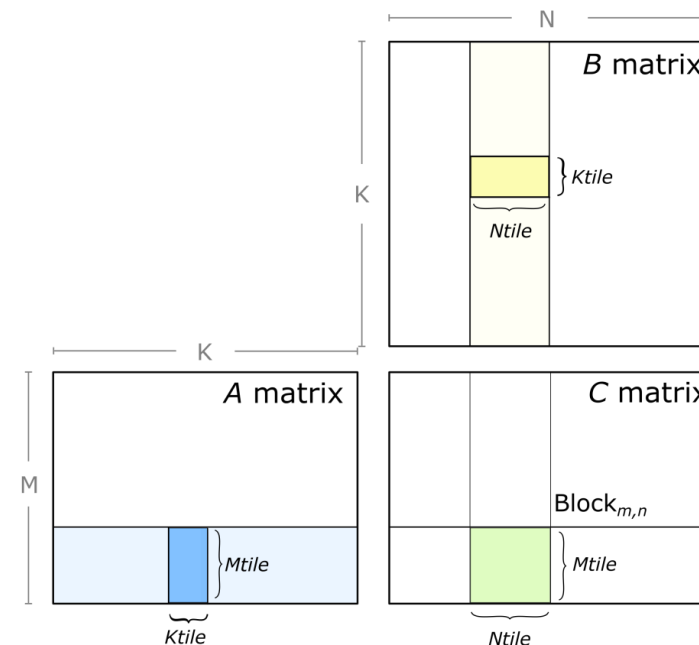
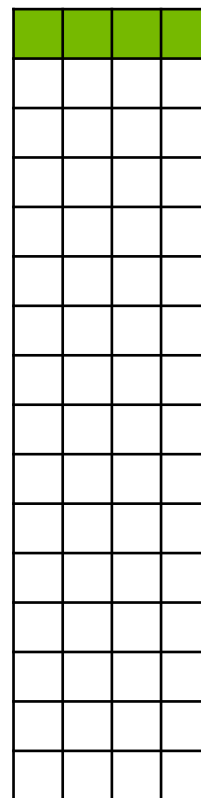
Thread Tile:

Block (32, 8) loads A tile (128, 8), B tile (8, 128) and compute on (128, 128) tile

Assume block size (32, 8), grid size (32, 32)

Each thread will compute (16, 4) C-sub-matrix.

Performance= 8642.93 GFlop/s, Time= 7.951 msec

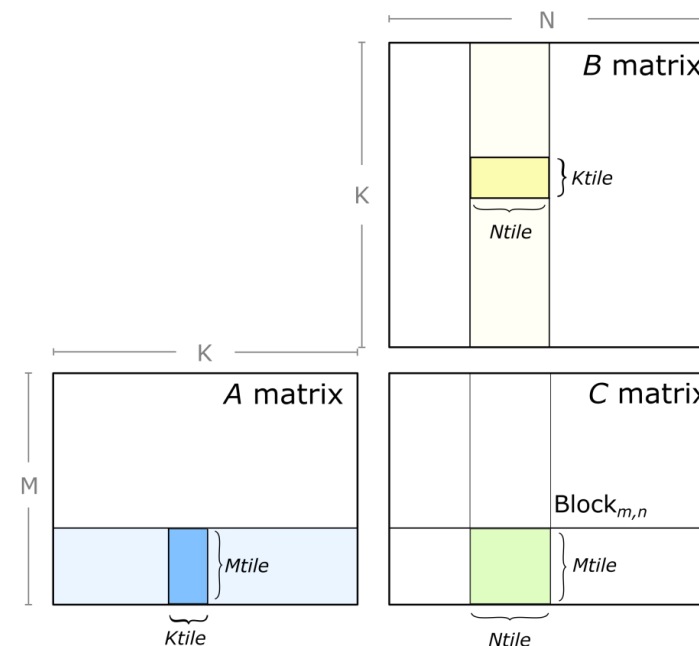
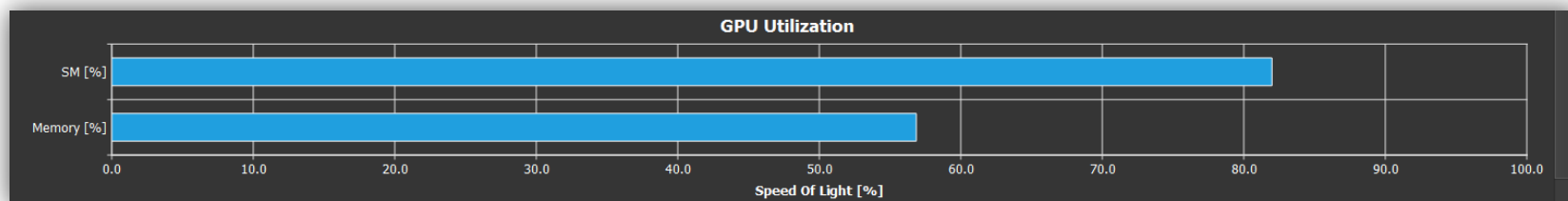
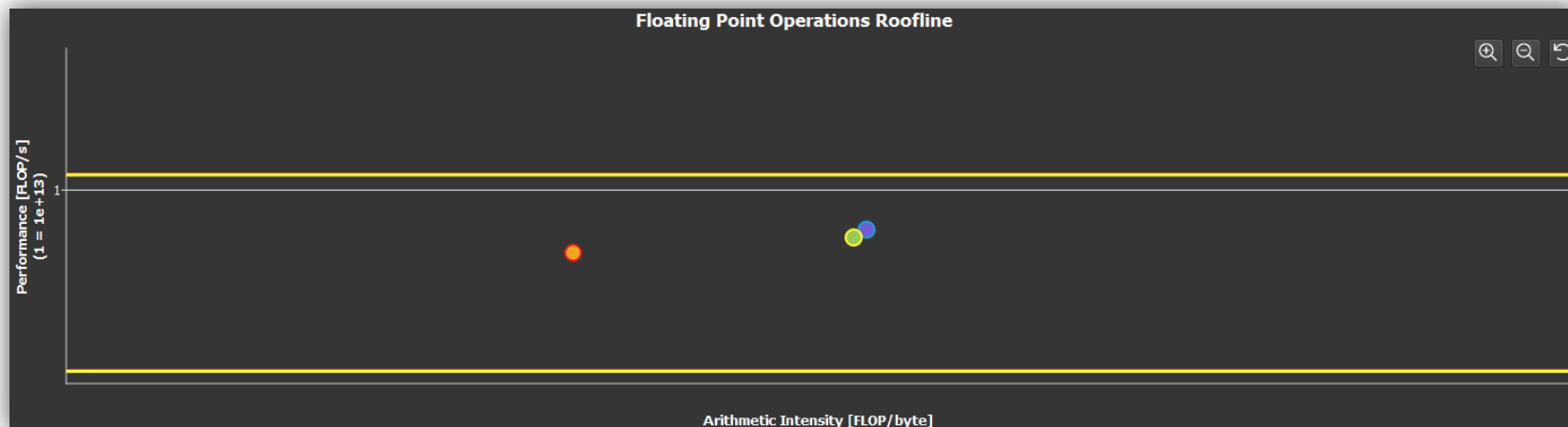


GEMM OPTIMIZATION STAGE 1

3.1 && 3.2

Thread Tile:

Increased Arithmetic Intensity (Flops/byte)



GEMM OPTIMIZATION STAGE 1

4. Swizzle and float4

Swizzle:

Block (32, 8) loads A tile (8, 128), B tile (8, 128) and compute on (128, 128) tile

Assume block size (32, 8), grid size (32, 32)

Each thread will compute (8, 8) C-sub-matrix

Transform 8 warps, 32 threads into new shape in order to achieve higher bandwidth, less memory loading.

- **Accumulated outer products:** LDS, only loading Asub col and Bsub row once.
- **Float4:** one LDS.128 instruction
- **LDS.U.128:** double bandwidth

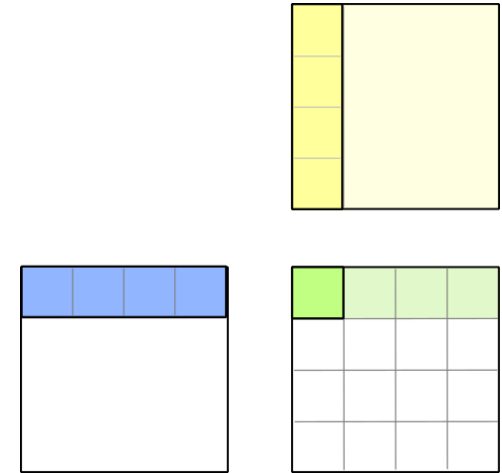
Accumulated outer products

Inner Product

For each thread, it will load values from shared memory.

Compute independent dot products

```
// Independent dot products
for (int i = 0; i < M; ++i)
    for (int j = 0; j < N; ++j)
        for (int k = 0; k < K; ++k)
            C[i][j] += A[i][k] * B[k][j];
```

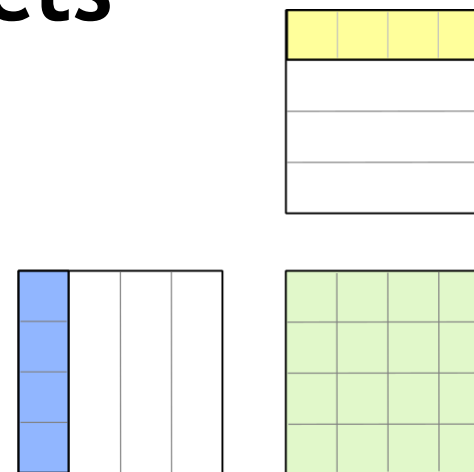


Inefficient due to large working sets to hold parts of A and B

Accumulated outer products

Accumulated outer products

We can load A col and B row iteratively, and each time it will compute a 4 * 4 matrix instead of one val.



Compute independent dot products

```
// Independent dot products
for (int i = 0; i < M; ++i)
    for (int j = 0; j < N; ++j)
        for (int k = 0; k < K; ++k)
            C[i][j] += A[i][k] * B[k][j];
```

Permute loop nests

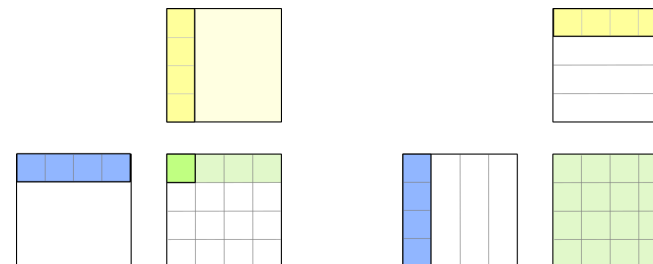
```
// Accumulated outer products
for (int k = 0; k < K; ++k)
    for (int i = 0; i < M; ++i)
        for (int j = 0; j < N; ++j)
            C[i][j] += A[i][k] * B[k][j];
```

Accumulated outer products

If kTile has the size of 32, each thread reads A_shared and B_shared in the stride of 32 floats (not float4 optimization) and compute a 4 x 4 sub matrix.

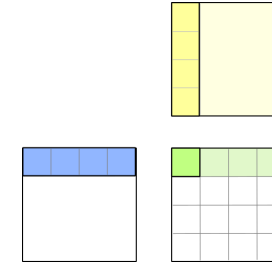
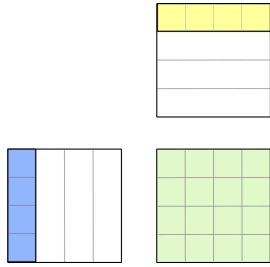
Should we have 1/4 LDS.32 in Outer Product compared with Inner Product ?

- Inner Product: LDS32 instructions: $4 \times 4 \times 32 \times 2 = 1024$?
- Outer Product: LDS 32 instructions $32 \times 4 \times 2 = 256$



```
__inline__ __device__ void calcInnerBlockByOuterProduct(float Csub[elemPerTy][elemPerTx], float *ptrAs, float *ptrBs)
{
    #pragma unroll
    for (int k = 0; k < kBlock; ++k)
    {
        #pragma unroll
        for (int i = 0; i < elemPerTy; i++) {
            #pragma unroll
            for (int j = 0; j < elemPerTx; j++) {
                Csub[i][j] += ptrAs[k * mBlock + i * 32] * ptrBs[k * nBlock + j * 32];
            }
        }
    }
}
```

Accumulated outer products



```
109
110 __inline__ __device__ void calcInnerBlockByOuterProduct(float Csub[elemPerTy][elemPerTx], float *ptrAs, fl
111 {
112     #pragma unroll
113     for (int k = 0; k < kBlock; k++)
114     {
115         #pragma unroll
116         for (int i = 0; i < elemPerTy; i++)
117         {
118             #pragma unroll
119             for (int j = 0; j < elemPerTx; j++)
120             {
121                 Csub[i][j] += ptrAs[k * mBlock + i * 32] * ptrBs[k * nBlock + j * 32];
122             }
123         }
124     }
125 }
```

Shared(256)	Load(256)	32(256)
-------------	-----------	---------

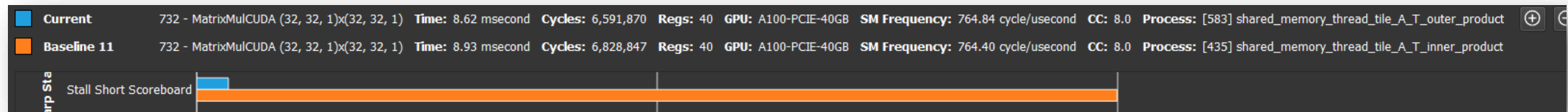
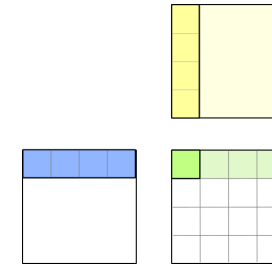
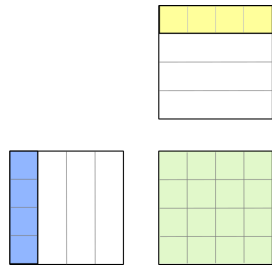
Outer Product: 256 LDS 32

```
110 __inline__ __device__ void calcInnerBlockByInnerProduct_float4(float Csub[elemPerTy][elemPerTx], float *pt
111 {
112     #pragma unroll
113     for (int i = 0; i < elemPerTy; i++)
114     {
115         #pragma unroll
116         for (int j = 0; j < elemPerTx; j++)
117         {
118             #pragma unroll
119             for (int k = 0; k < kBlock; k++)
120             {
121                 Csub[i][j] += (ptrAs + k * mBlock + i)[0] * (ptrBs + k * mBlock + j)[0];
122             }
123         }
124     }
125 }
```

Shared(256)	32(256)	Load(256)
-------------	---------	-----------

Inner Product: Also 256 LDS 32

Accumulated outer products



Outer Product: longer distance between FMA and its needed LDS →

smaller stall short scoreboard (4 cycles → 0.14 cycle)

This works the same for float4 (or unrolling consecutive floats)

Accumulated outer products

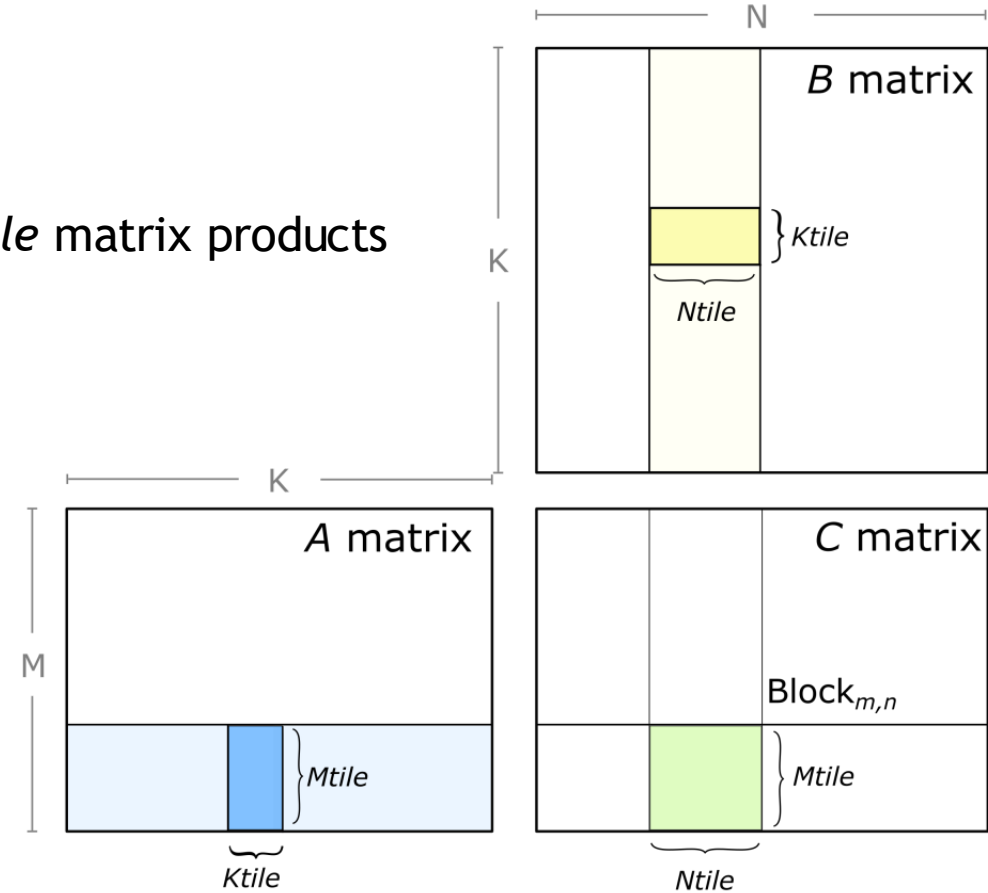
Computing matrix product one block at a time

Partition the loop nest into *blocks* along each dimension

- Partition into *Mtile*-by-*Ntile* independent matrix products
- Compute each product by accumulating *Mtile*-by-*Ntile*-by-*Ktile* matrix products

```
for (int mb = 0; mb < M; mb += Mtile)
  for (int nb = 0; nb < N; nb += Ntile)
    for (int kb = 0; kb < K; kb += Ktile)
    {
      // compute Mtile-by-Ntile-by-Ktile matrix product
      for (int k = 0; k < Ktile; ++k)
        for (int i = 0; i < Mtile; ++i)
          for (int j = 0; j < Ntile; ++j)
          {
            int row = mb + i;
            int col = nb + j;

            C[row][col] +=
              A[row][kb + k] * B[kb + k][col];
          }
    }
```



FLOAT4

LDG, STS, LDG, STG

4 LDG 32 Coalesced Global Memory Access

4 STS: no bank conflicts

# Source	Memory Address Space	Memory Access Operation	Memory Access Size
93 __inline__ __device__ void loadAnB_V1(float *As, float *Bs, float *ptrA, float *ptrB)			
94 {			
95 #pragma unroll			
96 for (int i = 0; i < mBlock; i += 32)			
97 {			
98 As[i] = ptrA[i];	Global(4), Shared(4)	Load(4), Store(4)	32(8)
99 }			
100			
101 #pragma unroll			
102 for (int i = 0; i < nBlock; i += 32)			
103 {			
104 Bs[i] = ptrB[i];	Global(4), Shared(4)	Load(4), Store(4)	32(8)
105 }			
106 }			

FLOAT4

LDG, STS, LDG, STG

LDG 128 Coalesced Global Memory Access

STS 128

- Reduce number of instructions
- LDG Sectors and STS Wavefronts → Discussion

```
73 // move AnB from global memory to shared memory and calculate mask for A
74 __inline__ __device__ void loadAnB(float *As, float *Bs, float *ptrA, float *ptrB)
75 {
76     *(float4 *) As = *(float4 *) ptrA;
77     *(float4 *) Bs = *(float4 *) ptrB;
78 }
79
```

# Source	Memory Access Operation	Memory Address Space	Memory Access Size
53 // Loop over all the sub-matrices of A and B required to compute the block sub-matrix			
54 for (int gBlock = 0; gBlock < K; gBlock += kBlock) {			
55			
56 loadAnB(ptrAs, ptrBs, ptrA, ptrB);	Load(2), Store(2)	Global(2), Shared(2)	128(4)

FLOAT4

LDG, STS, LDG, STG

4B or smaller words:

- Process addresses of all threads in a warp in a single phase

8B words are accessed in 2 phases:

- Process addresses of the **first 16** threads in a warp
- Process addresses of the **second 16** threads in a warp

16B words are accessed in 4 phases:

- Each phase processes a quarter of a warp

Bank conflicts occur only between threads in the same phase

LDS 128

- Reduce number of instructions
- Bandwidth: 4 wavefronts ?

#	Source	Memory Access Operation	Memory Address Space	Memory Access Size
87	<code>*(float4 *)arows = *(float4 *)(ptrAsSwizzle + k * mBlock);</code>	Load(8)	Shared(8)	128(8)
88	<code>*(float4 *)(arows + 4) = *(float4 *)(ptrAsSwizzle + k * mBlock + 32);</code>	Load(8)	Shared(8)	128(8)
89	<code>*(float4 *)bcols = *(float4 *)(ptrBsSwizzle + k * nBlock);</code>	Load(8)	Shared(8)	128(8)
90	<code>*(float4 *)(bcols + 4) = *(float4 *)(ptrBsSwizzle + k * nBlock + 16);</code>	Load(8)	Shared(8)	128(8)

FLOAT4

LDG, STS, LDG, STG

STG 128

- Reduce number of instructions
- May reduce number of written sectors (uncoalesced memory access)

# Source	Memory Access Operation	Memory Address Space	Memory Access Size a
77 #pragma unroll			
78 for (int j = 0; j < 2; j++) {			
79 *(float4 *)(ptrD + (32 * i + 0) * N + j * 16) = *(float4 *)&Csub[i * 4 + 0][j * 4];	Store(4)	Global(4)	128(4)
80 *(float4 *)(ptrD + (32 * i + 1) * N + j * 16) = *(float4 *)&Csub[i * 4 + 1][j * 4];	Store(4)	Global(4)	128(4)
81 *(float4 *)(ptrD + (32 * i + 2) * N + j * 16) = *(float4 *)&Csub[i * 4 + 2][j * 4];	Store(4)	Global(4)	128(4)
82 *(float4 *)(ptrD + (32 * i + 3) * N + j * 16) = *(float4 *)&Csub[i * 4 + 3][j * 4];	Store(4)	Global(4)	128(4)
83 }			
84 }			
85 }			

THREAD BLOCK TILE STRUCTURE

Parallelism Within a CUDA Thread Block

Decompose thread block into warp-level tiles

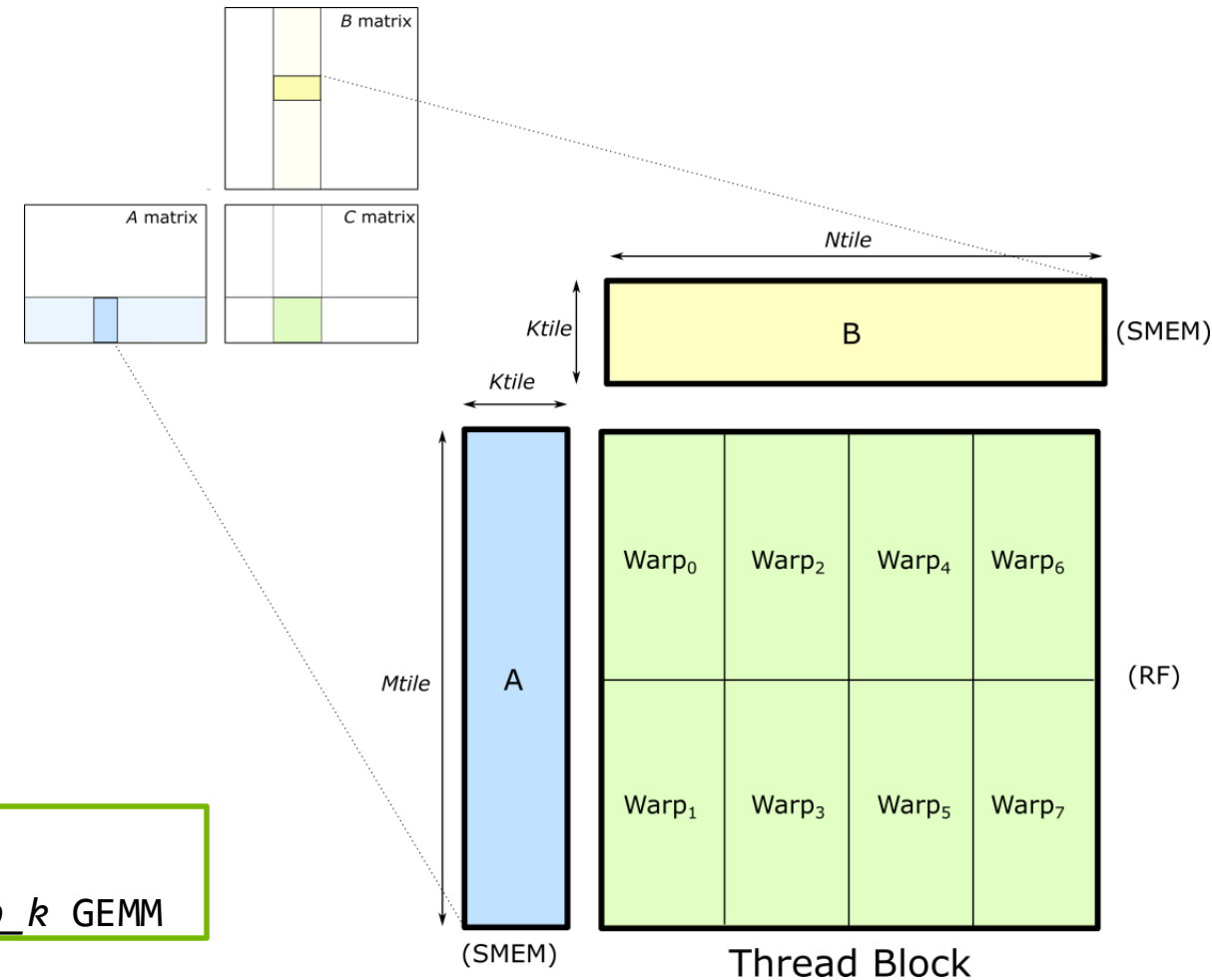
- Load **A** and **B** operands into Shared Memory (**reuse**)
- **C** matrix distributed among warps

Each warp computes an independent matrix product

```
for (int kb = 0; kb < K; kb += Ktile)
{
    .. load A and B tiles to shared memory

    for (int m = 0; m < Mtile; m += warp_m)
        for (int n = 0; n < Ntile; n += warp_n)
            for (int k = 0; k < Ktile; k += warp_k)
                .. compute warp_m by warp_n by warp_k GEMM
}
```

by each CUDA warp



WARP-LEVEL TILE STRUCTURE

Warp-level matrix product

Warps perform an accumulated matrix product

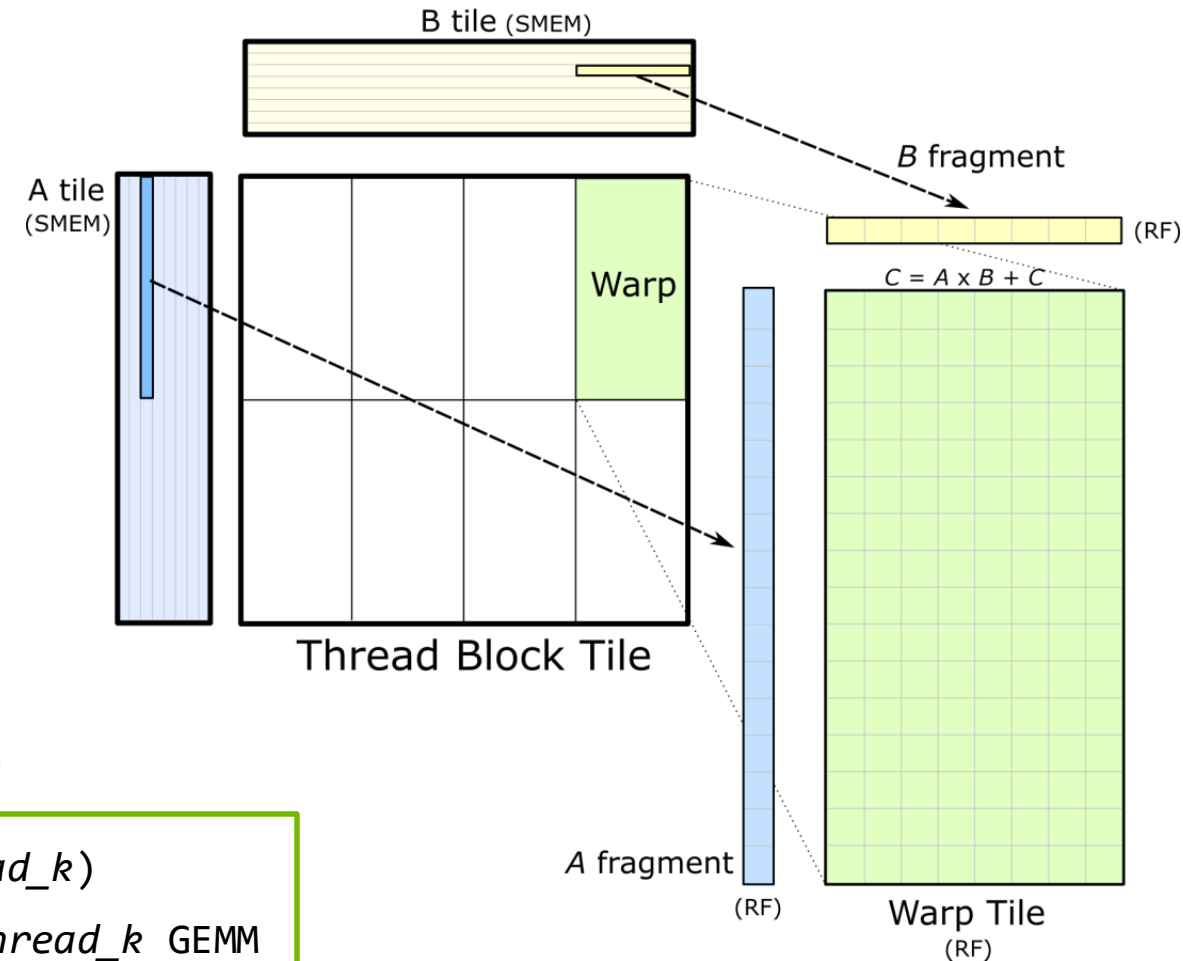
- Load *A* and *B* operands from SMEM into registers
- *C* matrix held in registers of participating threads

Shared Memory layout is *K*-strided for efficient loads

```
for (int k = 0; k < Ktile; k += warp_k)
{
    .. load A tile from SMEM into registers
    .. load B tile from SMEM into registers

    for (int tm = 0; tm < warp_m; tm += thread_m)
        for (int tn = 0; tn < warp_n; tn += thread_n)
            for (int tk = 0; tk < warp_k; tk += thread_k)
                .. compute thread_m by thread_n by thread_k GEMM
}
```

by each CUDA thread



THREAD-LEVEL TILE STRUCTURE

Parallelism within a thread

Threads compute accumulated matrix product

- A , B , and C held in registers

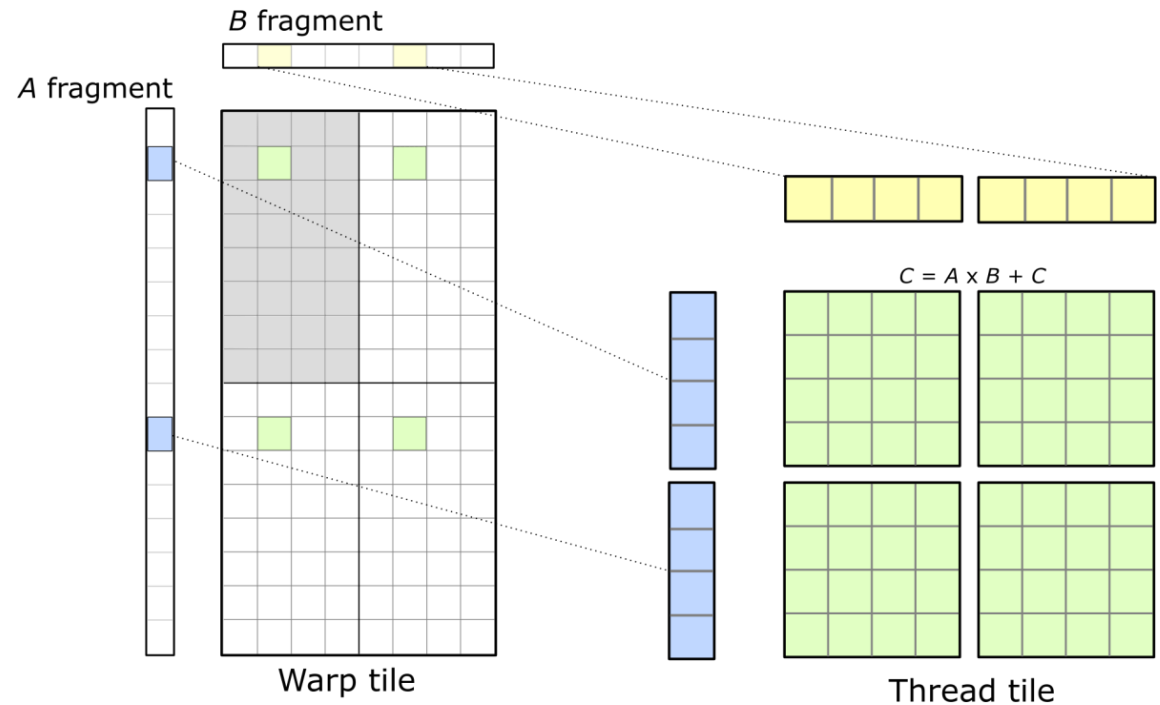
Opportunity for data reuse:

- $O(M*N)$ computations on $O(M+N)$ elements

```
for (int m = 0; m < thread_m; ++m)
  for (int n = 0; n < thread_n; ++n)
```

```
    for (int k = 0; k < thread_k; ++k)
      C[m][n] += A[m][k] * B[n][k];
```

Fused multiply-accumulate instructions



LDS.U WITH DOUBLE BANDWIDTH

4. Swizzle and float 4

Each SMEM fetch value is shared by 4-8 threads

double effective SMEM bandwidth for “uniform addresses”. Apply to **Maxwell and Later Architecture**

Two supported address patterns:

- ▶ $\text{Addr}[\text{tid}] == \text{Addr}[\text{tid}^1]$
(each thread and its immediate neighbor are sharing values)
- ▶ $\text{Addr}[\text{tid}] == \text{Addr}[\text{tid}^2]$

This applies to both **LDS.64** and **LDS.128**

column of A
in SMEM

lane id

row of B
in SMEM

0	1	16	17
2	3	18	19
4	5	20	21
6	7	22	23
8	9	24	25
10	11	26	27
12	13	28	29
14	15	30	31

A access
 $\text{addr}[\text{tid}] = \text{addr}[\text{tid}^1]$

0	1	16	17
2	3	18	19
4	5	20	21
6	7	22	23
8	9	24	25
10	11	26	27
12	13	28	29
14	15	30	31

B access
 $\text{addr}[\text{tid}] = \text{addr}[\text{tid}^2]$

https://wiki.nvidia.com/gpuhwkepler/index.php/Compute/Application_Performance/Applications/Linear_Algebra/GEMM/GEMM_for_Dummies

KERNEL GRID STRUCTURE

4. Swizzle

1. Store A_{sub} and B_{sub} from Global memory to Shared Memory

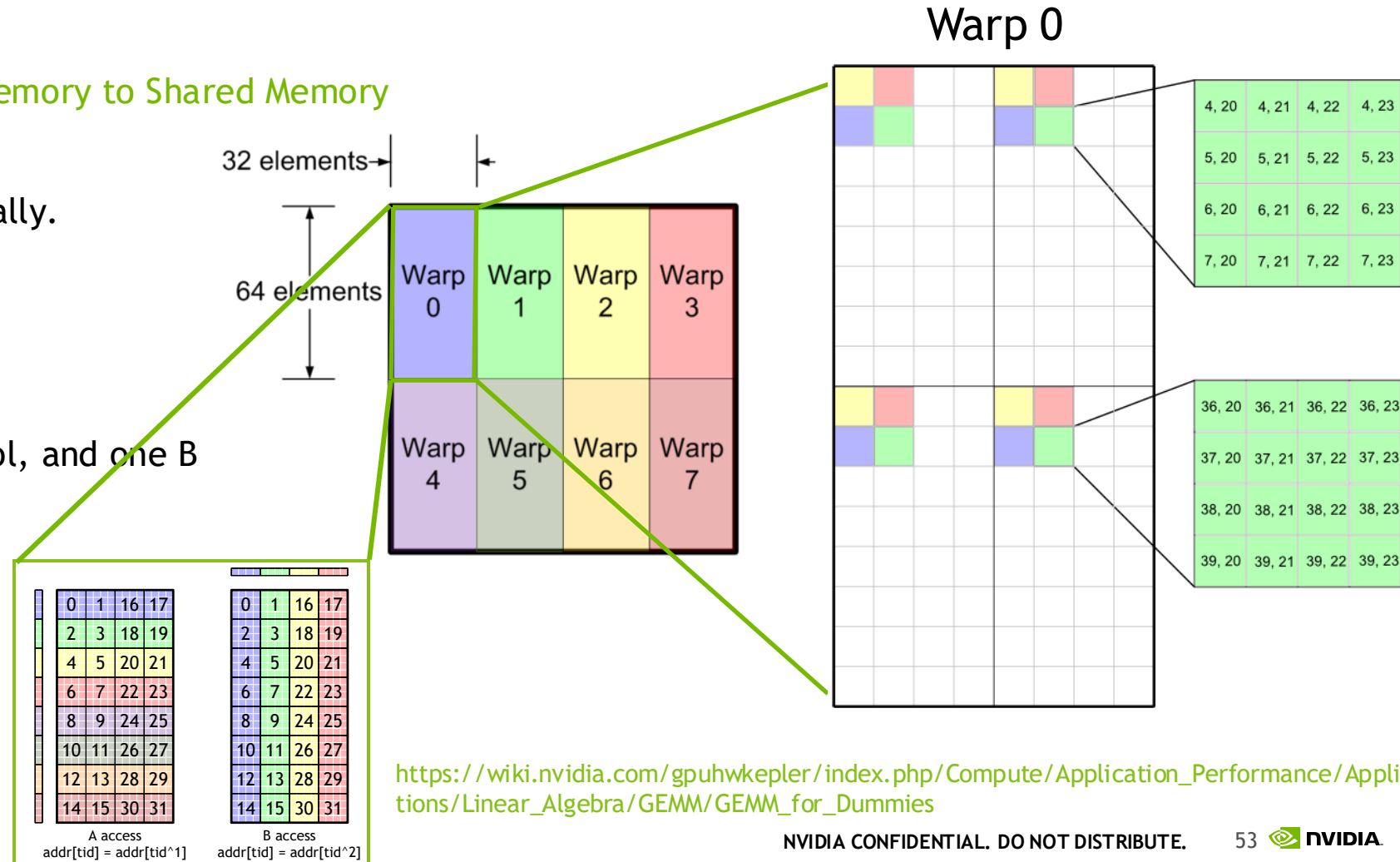
Warp and threads are shaped normally.

2. Compute on 2 x 2 x (4, 4) matrix

(4, 4) matrix LDS.U.128 one A sub col, and one B
Sub row, then do outer product.

3. Store result to global memory

Store 2 x 2 x (4, 4)



https://wiki.nvidia.com/gpuhwkepler/index.php/Compute/Application_Performance/Applications/Linear_Algebra/GEMM/GEMM_for_Dummies

GEMM OPTIMIZATION STAGE 1

4. Swizzle and float4

LDS.U.128

Block size (32, 8), grid size (32, 32)

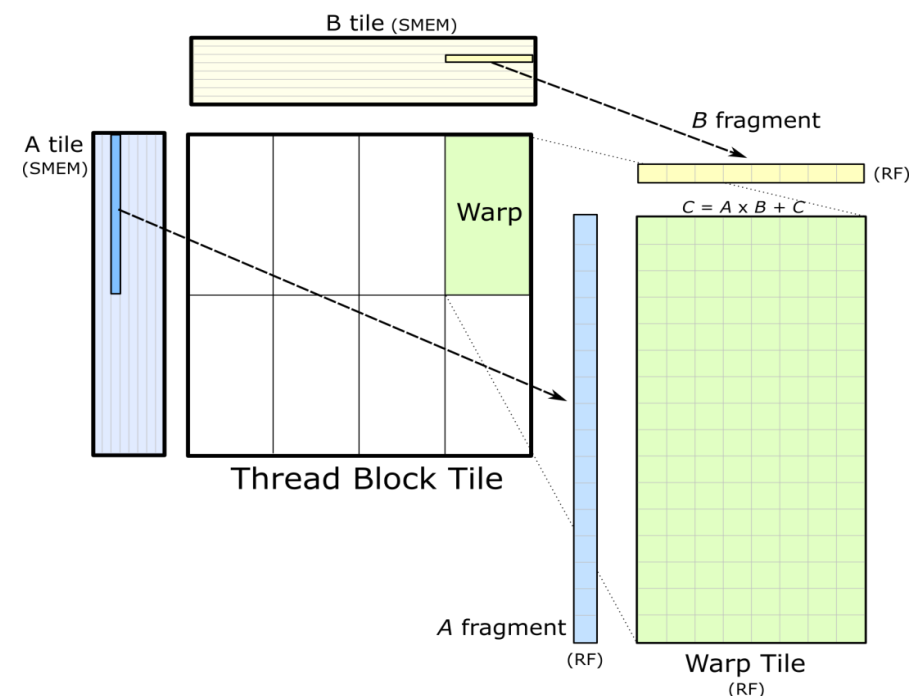
Outer k iterations: $2048/8 = 256$; Inner k iterations: 8;

Number of Instructions (LDS.128)

Each thread in each inner k iteration needs LDS.128 (8)

So LDS instructions = $32 \times 32 \times 256 \times 8 \times 8 \times 4 = 67,108,864$

Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	66,846,720 (+0.00%)	66,846,720 (+0.00%)	133,717,785 (+0.00%)	21.67 (+0.00%)	17,422 (+0.00%)
Shared Load Matrix					



Num of Instructions = $\text{blockDim.x} \times \text{blockDim.y} \times \text{Num_Kblocks} \times \text{Num_warps_per_block} \times \text{Num_Instruction_per_warp}$

GEMM OPTIMIZATION STAGE 1

4. Swizzle and float4

LDS.U.128

Block size (32, 8), grid size (32, 32)

Outer k iterations: $2048/8 = 256$; Inner k iterations: 8;

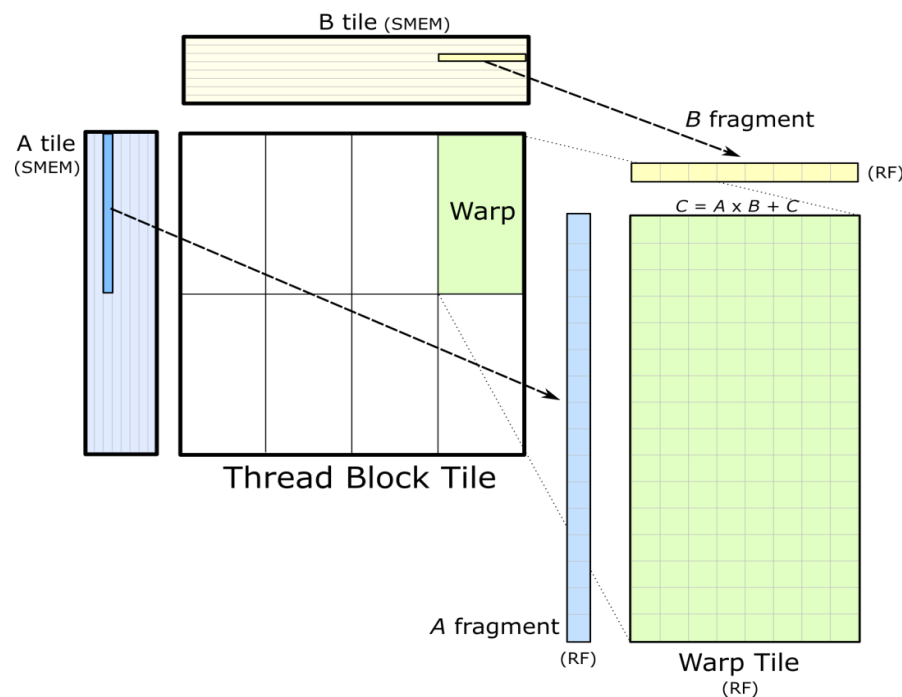
Number of wavefronts (LDS.128)

Each thread in each inner k iteration needs LDS.128 (8)

LDS.U doubles bandwidth (2 wavefronts)

LDS wavefronts = number of instructions x 2 = 134, 217, 728

Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	66,846,720 (+0.00%)	66,846,720 (+0.00%)	133,717,785 (+0.00%)	21.67 (+0.00%)	17,422 (+0.00%)
Shared Load Matrix					



Num of Instructions = $\text{blockDim.x} \times \text{blockDim.y} \times \text{Num_Kblocks} \times \text{Num_warps_per_block} \times \text{Num_Instruction_per_warp}$

GEMM OPTIMIZATION STAGE 1

4. Swizzle by row or col and float4

LDS.U.128

LDS.U doubles bandwidth (2 wavefronts) either for LDS A sub or LDS B sub

Can not double both LDS.

LDS instructions = $32 \times 32 \times 256 \times 8 \times 8 \times 4 = 67,108,864$

LDS wavefronts = $32 \times 32 \times 256 \times 8 \times 8 \times (2 \times 2 + 2 \times 4) = 201,326,592$

Shared Memory						
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts	
Shared Load	67,108,864 (+0.39%)	67,108,864 (+0.39%)	201,356,184 (+50.58%)	32.05 (+47.91%)	21,984 (+26.19%)	
Shared Load Matrix						
Shared Store	4,194,304 (+0.39%)	4,194,304 (+0.39%)	16,777,216 (+0.39%)	0.67 (-1.39%)	1,176,599 (+11.43%)	

Shared Memory						
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts	
Shared Load	67,108,864 (+0.39%)	67,108,864 (+0.39%)	201,354,535 (+50.58%)	31.72 (+46.39%)	19,968 (+14.61%)	
Shared Load Matrix						
Shared Store	4,194,304 (+0.39%)	4,194,304 (+0.39%)	16,777,216 (+0.39%)	0.66 (-2.40%)	1,064,699 (+0.84%)	

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19
20	21	22	23
24	25	26	27
28	29	30	31

Warp reshape in row major

0	8	16	24
1	9	17	25
2	10	18	26
3	11	19	27
4	12	20	28
5	13	21	29
6	14	22	30
7	15	23	31

Warp reshape in col major

GEMM OPTIMIZATION STAGE 1

Final Process

```
// Loop over all the sub-matrices of A and B required to compute the block sub-matrix
for (int gBlock = 0; gBlock < K; gBlock += kBlock) {

    loadAnB(ptrAs, ptrBs, ptrA, ptrB);
    ptrA += aStep;
    ptrB += bStep;

    // Synchronize to make sure the matrices are loaded
    __syncthreads();

    calcInnerBlock(Csub, ptrAsSwizzle, ptrBsSwizzle);

    // Synchronize to make sure that the preceding
    // computation is done before loading two new
    // sub-matrices of A and B in the next iteration
    __syncthreads();
}

// Write the block sub-matrix to device memory; each thread writes 2*2*4*4 element
float * ptrD = C + (blockRow * mBlock + warp_y + y_swizzle * 4) * N
                + (blockCol * nBlock + warp_x + x_swizzle * 4);

#pragma unroll
for (int i = 0; i < 2; i++) {
    #pragma unroll
    for (int j = 0; j < 2; j++) {
        *(float4 *)(ptrD + (32 * i + 0) * N + j * 16) = *(float4 *)&Csub[i * 4 + 0][j * 4];
        *(float4 *)(ptrD + (32 * i + 1) * N + j * 16) = *(float4 *)&Csub[i * 4 + 1][j * 4];
        *(float4 *)(ptrD + (32 * i + 2) * N + j * 16) = *(float4 *)&Csub[i * 4 + 2][j * 4];
        *(float4 *)(ptrD + (32 * i + 3) * N + j * 16) = *(float4 *)&Csub[i * 4 + 3][j * 4];
    }
}
}
```

For each MK and NK tile

Load MK tile and NK tile
to Shared Memory
Block, warp shapes as normal

Outer Product Calculation
and Accumulation
Block, warp shapes as special swizzle

Store Result to Global Memory
Block, warp shapes as special swizzle

GEMM OPTIMIZATION STAGE 1

Final Perf

Step	Perf (GFlop/s)	Percentage of Peak Perf (%)	Time (ms)	Cuda Block Size	Corresponding C Block Size	Thread Tile Size	Improvement Percentage	Memory Techniques
Base Line	1861.22	17.63%	36.922	(32, 32)	(32, 32)	/	0.00%	No shared memory
shared memory	2862.98	27.11%	24.003	(32, 32)	(32, 32)	/	53.82%	A, B shared memory
shared memory + thread tile	7989.69	75.67%	8.601	(32, 32)	(128, 128)	(4, 4)	329.27%	A, B shared memory, Csub on Registers
Thread_Tile_8_4	8431.95	79.86%	8.15	(32, 16)	(128, 128)	(8, 4)	353.03%	A, B shared memory, Csub on Registers
Thread_Tile_16_4	8642.93	81.85%	7.951	(32, 8)	(128, 128)	(16, 4)	364.37%	A, B shared memory, Csub on Registers
Swizzle+Float4	9159.75	86.75%	7.502	(32, 8)	(128, 128)	(8, 8)	392.14%	A, B shared memory, Csub on Registers

GEMM OPTIMIZATION STAGE 2

Based on swizzle and float4

What we have done:

Block (32, 8) loads A tile (8, 128), B tile (8, 128) and compute on (128, 128) tile

Each thread computes 8*8 C elements in 4 4*4 contiguous blocks, each warp owns 64*32 contiguous C elements, each block owns 128*128 contiguous C elements

Outer loop

Float4

Thread id swizzling

What we are going to do:

avoid stalls on shared and global memory loads

BLOCKED GEMM

```
__global__ void GEMM(float *C, int M, int K, int N, const float *A_gmem, const float *B_gmem) {  
    __shared__ float Ash[ KblockSize ][ MblockSize ];  
    __shared__ float Bsh[ KblockSize ][ NblockSize ];  
  
    float Ctile[ ElementColsPerThread ][ ElementRowsPerThread ] = {0};  
    for (int kblock = 0; kblock < K; kblock += KblockSize ) {  
        Load block from A_gmem to Ash  
        Load block from B_gmem to Bsh  
        __syncthreads();  
  
        Load block from smem to reg  
        Ctile += Atile x Btile;  
        __syncthreads();  
    }  
  
    C[ .. ] = Ctile;  
}
```

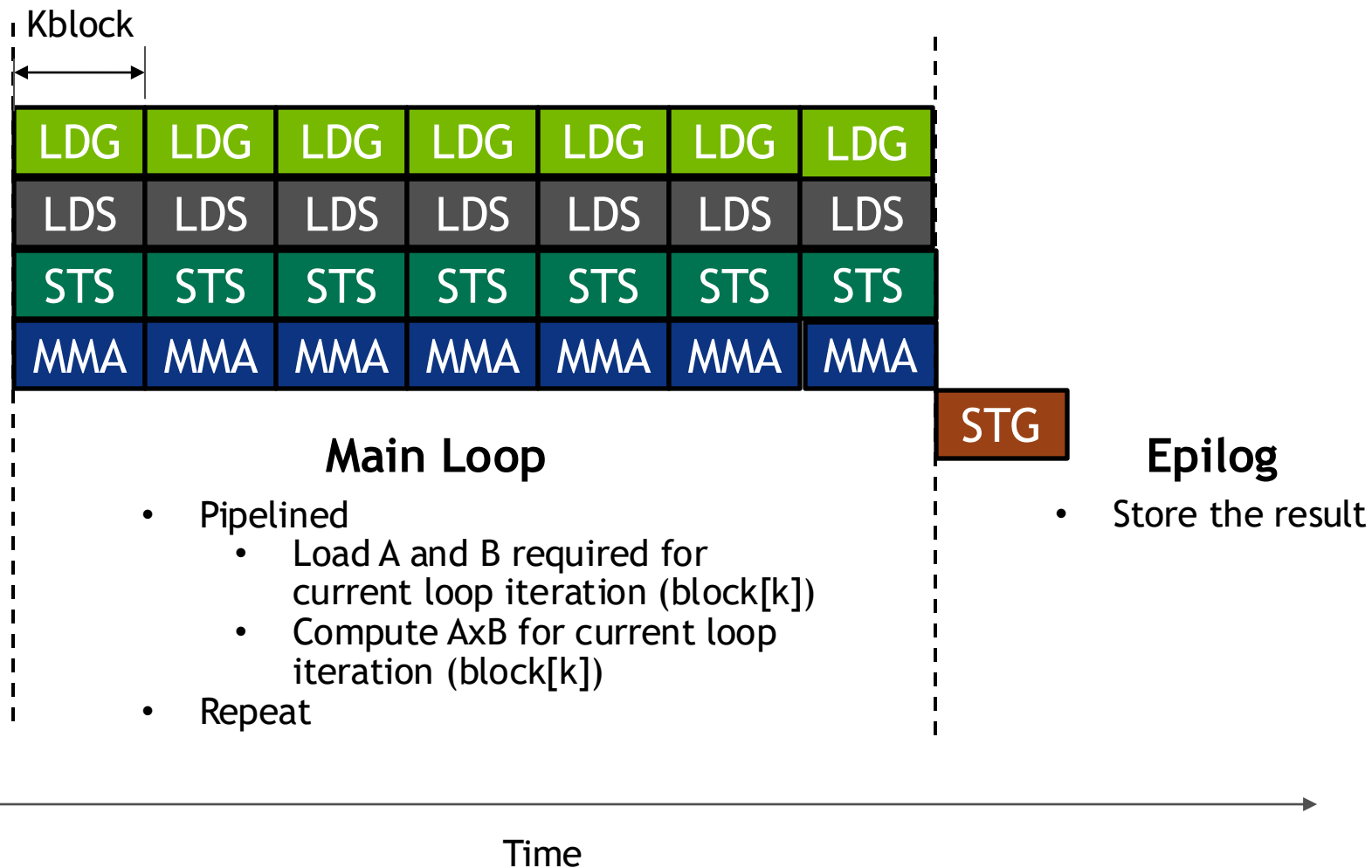
} from gmem to smem, LDG and STS block[k]

} accumulate outer product, LDS and FMA block[k]

} epilog: update C tile in global memory, STG

SOFTWARE PIPELINE

Blocked GEMM



AVOID STALL LONG SCOREBOARD

Prefetch data from global memory into shared memory

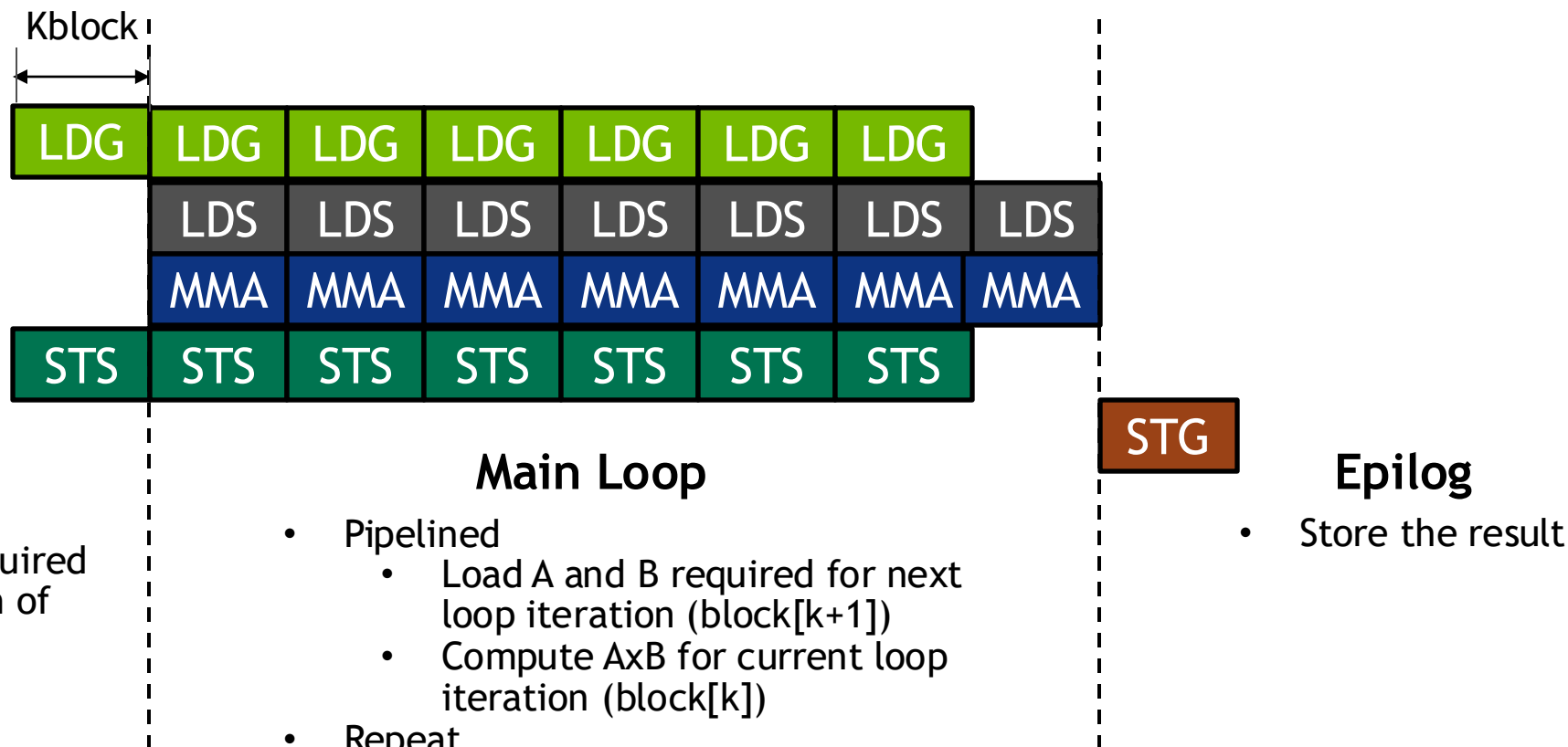
- ▶ Current perf: 9159.75 GFlop/s
- ▶ LDG and STS are put together, STS has to wait for the completion of LDG, which causes long scoreboard
- ▶ Key idea: Overlap LDG and FMA
- ▶ LDG-STS-FMA -> LDG-FMA-STS
- ▶ Restructure kernel to prefetch data (LDG)
 - ▶ Prefetch block[k+1] from gmem to reg
 - ▶ `__syncthreads`
 - ▶ Load block[k] from smem to reg and perform FFMA
 - ▶ `__syncthreads`
 - ▶ Prefetch block[k+1] from reg to smem (STS)

LDG.E.128 R88, [R96.64]	77	583	299
LDG.E.128 R92, [R98.64]	81	155	59
IADD3 R0, R0, 0x8, RZ	81	31	0
ISETP.GE.AND P0, PT, R0, c[0x0][0x180], PT	81	172	38
IMAD.WIDE R98, R3, 0x4, R98	81	1,017	474
STS.128 [R2.X16], R88	81	16,262	10,207
STS.128 [R2.X16+0x1000], R92	77		
BAR.SYNC 0x0	73		
LDS.128 R68, [R85.X4]	77		
LDS.128 R72, [R84+0x40]	81		

Total Sample Count: 16262
0.04% Mio Throttle (6)
0.06% Not Selected (10)
0.25% Selected (41)
99.65% Long Scoreboard (16205)

SOFTWARE PIPELINE

Prefetch data from global memory into shared memory



- Load A and B required for first iteration of loop (block[0])

- Pipelined
 - Load A and B required for next loop iteration (block[k+1])
 - Compute $A \times B$ for current loop iteration (block[k])
- Repeat

- Store the result

PREFETCH DATA FROM GMEM INTO SMEM

```
__global__ void GEMM(float *C, int M, int K, int N, const float *A_gmem, const float *B_gmem) {
```

```
    __shared__ float Ash[ kblockSize ][ MblockSize ];
```

```
    __shared__ float Bsh[ kblockSize ][ NblockSize ];
```

```
    float Ctile[ ElementColsPerThread ][ ElementRowsPerThread ] = {0};
```

Prolog	{	Load block[0] from gmem to smem	}	LDG and STS block[0]
Mainloop	{	for (int kblock = 0; kblock < K; kblock += kblockSize) {		
		Prefetch block[k+1] from gmem to reg	}	LDG block[k+1]
		__syncthreads();		
		Load block[k] from smem to reg	}	accumulate outer product, LDS and FMA block[k]
		Ctile += Atile x Btile;		
		__syncthreads();		
		Prefetch block[k+1] from reg to smem	}	STS block[k+1]
	}	}		
Epilog	{	C[..] = Ctile;	}	update C tile in global memory, STG
	}			

AVOID STALL LONG SCOREBOARD

Performance Analysis

- ▶ Current perf
 - ▶ 9720.78 GFlop/s, Time= 7.069 msec
- ▶ Speedup
 - ▶ $9720.78 / 9159.75 = 1.06x$
- ▶ Observation
 - ▶ Stall Long Scoreboard is avoided (0.97->0)
 - ▶ SOL SM is increased to 92.3%

Stall Long Scoreboard				
		Current	Baseline 2	
Stall IMC Miss	Stall Not Selected	2.44	2.11	15.78%
	Stall Math Pipe Throttle	2.40	2.09	14.59%
Stall Misc	Selected	1.00	1.00	0.00%
	Stall Wait	0.94	0.96	-2.42%
Stall Membar	Stall Barrier	0.27	0.29	-6.06%
	Stall Short Scoreboard	0.07	0.12	-44.24%
Stall Sleeping	Stall Dispatch Stall	0.02	0.00	777.27%
	Stall No Instruction	0.02	0.03	-21.10%
Stall Tex Throttle	Stall LG Throttle	0.01	0.02	-5.97%
	Stall MIO Throttle	0.01	0.02	-58.67%
	Stall Drain	0.00	0.00	-9.45%
	Stall Long Scoreboard	0.00	0.97	-99.70%
Instruction Statistics				
Statistics of the executed low-level instructions, while others remain unused in calls.	Stall IMC Miss	0.00	0.00	-1.68%
	Stall Misc	0.00	0.00	inf%
	Stall Membar	0.00	0.00	0.00%
	Stall Sleeping	0.00	0.00	0.00%
	Stall Tex Throttle	0.00	0.00	0.00%
Total Instructions [inst]				
Total Instructions [inst]				

AVOID STALL LONG SCOREBOARD

What else we can optimize

- ▶ We use two `__syncthreads()` in the mainloop. The first one is before LDS block[k] while the other is before STS block[k+1].

#	Address	Source	Live Registers	Sampling Data (All)	Sampling Data (Not Issued)
114	00007fe3 28fac910	LDG.E.128 R28, [R102.64]	86	187	51
115	00007fe3 28fac920	IADD3 R95, R95, 0x8, RZ	86	45	0
116	00007fe3 28fac930	BAR.SYNC 0x0	86	131	30
117	00007fe3 28fac940	LDS.128 R32, [R8.X4]	90	1,932	949

} 1st `__syncthreads()`

567	00007fe3 28fae560	LDS.128 R40, [R18+0xe00]	120	79	42
568	00007fe3 28fae570	FFMA R122, R5, R51, R68	121	31	0
569	00007fe3 28fae580	FFMA R120, R6, R51, R78	122	70	32
570	00007fe3 28fae590	BAR.SYNC 0x0	122	56	16
571	00007fe3 28fae5a0	STS.128 [R96.X16], R24	122	3,688	1,736
572	00007fe3 28fae5b0	FFMA R78, R57, R51, R86	118		0
573	00007fe3 28fae5c0	FFMA R68, R58, R51, R3	118		883
574	00007fe3 28fae5d0	STS.128 [R96.X16+0x1000], R28	118		8
575	00007fe3 28fae5e0	FFMA R17, R4, R51, R17	114		0
576	00007fe3 28fae5f0	FFMA R118, R7, R51, R16	115		33
577	00007fe3 28fae600	FFMA R0, R56, R51, R0	115		34

} 2nd `__syncthreads()`

Total Sample Count: 3688
0.05% Dispatch Stall (2)
0.08% Mio Throttle (3)
0.76% Selected (28)
1.30% Not Selected (48)
4.77% Wait (176)
93.03% Barrier (3431)

REMOVE ONE `__SYNCTHREADS()`

Shared memory double-buffering

- ▶ 2nd `__syncthreads()` is used to make sure LDS block[k] is finished before STS block[k+1] since they use the same smem buffer
- ▶ Key ideas:
 - ▶ Use double buffer of shared memory
 - ▶ Remove the dependency of LDS[k] and STS[k+1] by using different buffer of shared memory
 - ▶ Remove the 2nd `__syncthreads()`

```
for (int kblock = 0; kblock < K; kblock += KblockSize ) {
```

```
    Prefetch block[k+1] from gmem to reg  
    __syncthreads();
```

```
    Load block[k] from smem to reg  
    Ctile += Atile x Btile;
```

LDS block[k]

```
    __syncthreads();
```

2nd `__syncthreads`

```
    Prefetch block[k+1] from reg to smem
```

STS block[k+1]

```
}
```

SHARED MEMORY DOUBLE-BUFFERING

```
__global__ void GEMM(float *C, int M, int K, int N, const float *A_gmem, const float *B_gmem) {  
    __shared__ float Ash[ 2*kblockSize ][ MblockSize ];  
    __shared__ float Bsh[ 2*kblockSize ][ NblockSize ];  
    int smem_buffer_idx = 0;  
  
    float Ctile[ ElementColsPerThread ][ ElementRowsPerThread ] = {0};  
    Load block[0] from gmem to smem[0]  
    for (int kblock = 0; kblock < K; kblock += KblockSize ) {  
        Prefetch block[k+1] from gmem to reg  
        __syncthreads();  
  
        Load block[k] from smem[smem_buffer_idx] to reg  
        Ctile += Atile x Btile;  
        __syncthreads();  
  
        smem_buffer_idx ^= 1;  
        Prefetch block[k+1] from reg to smem[smem_buffer_idx]  
    }  
  
    c[ .. ] = Ctile;  
}
```

} LDG and STS block[0]

} LDG block[k+1]

} accumulate outer product
LDS and FMA block[k]

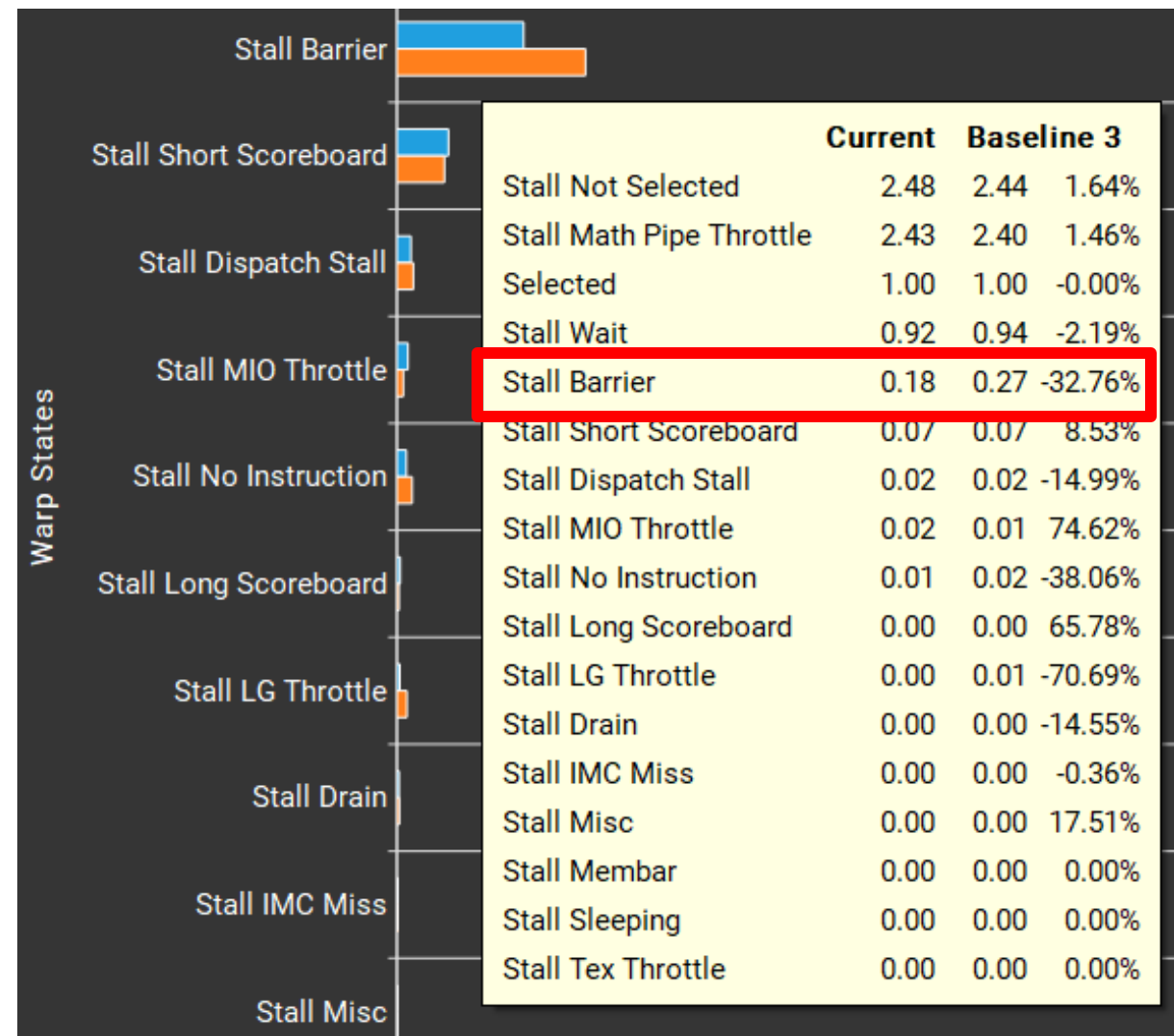
} STS block[k+1]

} update C tile in global memory, STG

REMOVE ONE __SYNCTHREADS()

Performance Analysis

- ▶ Current perf
 - ▶ 9705.09 GFlop/s, Time= 7.081 msec
- ▶ Speedup
 - ▶ $9705.09 / 9720.78 = 0.998x$
- ▶ Observation
 - ▶ Stall Barrier is reduced (0.27->0.18)
 - ▶ SOL SM is a little decreased to 92.15%



REMOVE ONE __SYNCTHREADS()

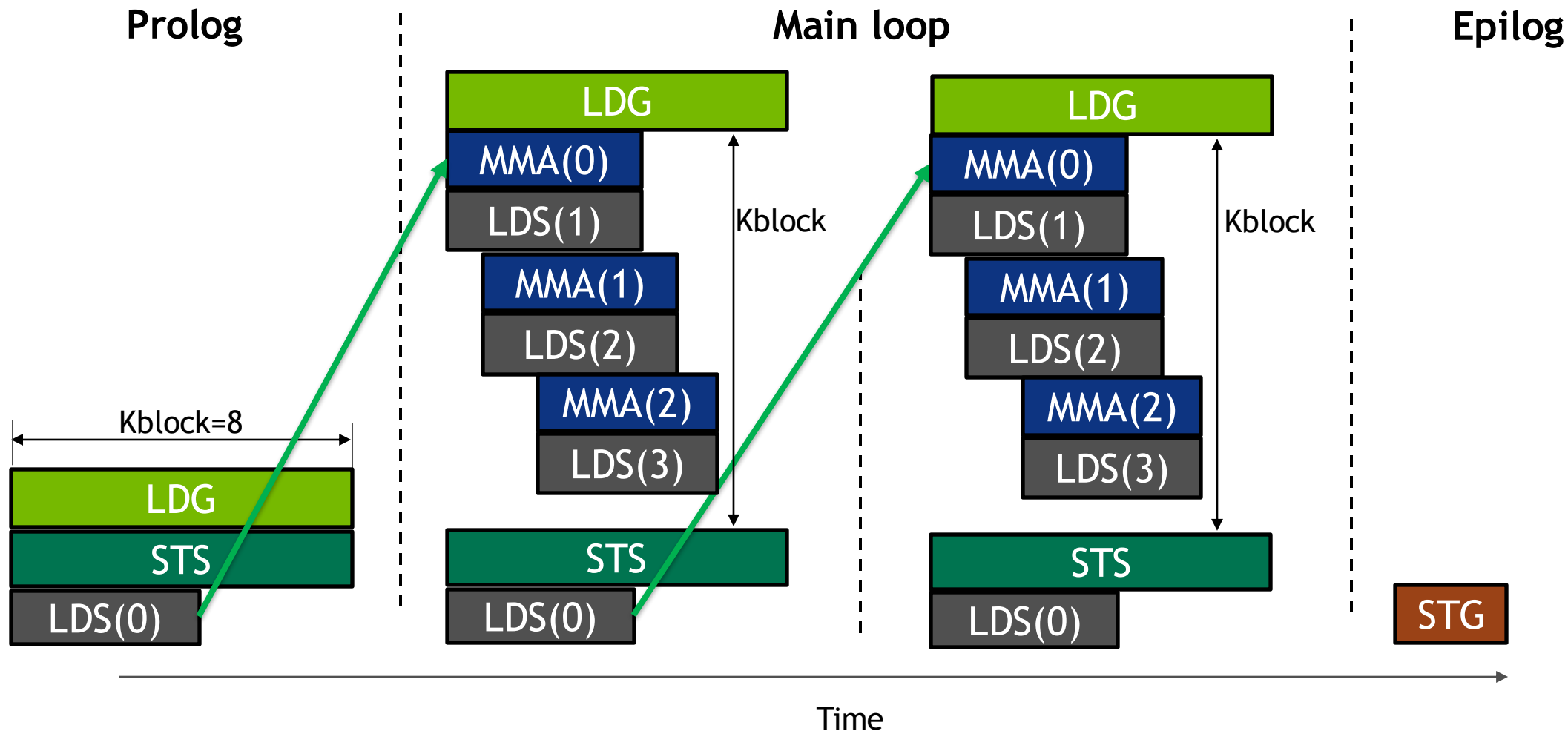
What else we can optimize

- ▶ LDS block[k-1] and FMA block[k-1] is put together. FMA block[k-1] has to wait for the completion of LDS block[k-1].
- ▶ If LDS block[k-1] and FMA block[k-1] are split in the pipeline, then Stall Short Scoreboard can be reduced.

#	Address	Source	Live Registers	Sampling Data (All)	Sampling Data (Not Issued)
123	00007fb3 68fac9a0	IMAD.WIDE R102, R87, 0x4, R102	94	197	30
124	00007fb3 68fac9b0	LDS.128 R40, [R2.X4+UR4+0x80]	98	144	88
125	00007fb3 68fac9c0	LDS.128 R44, [R3.X4+UR4+0x2000]	102	227	85
126	00007fb3 68fac9d0	FFMA R101, R32, R39, R54	103	3,994	1,918
127	00007fb3 68fac9e0	FFMA R104, R33, R39, R48	104	Total Sample Count: 3994 0.08% Dispatch Stall (3) 0.75% Selected (30) 30.85% Short Scoreboard (1232) 33.95% Not Selected (1356) 34.38% Math Pipe Throttle (1373)	
128	00007fb3 68fac9f0	FFMA R105, R34, R39, R49	105		
129	00007fb3 68faca00	FFMA R106, R35, R39, R50	106		
130	00007fb3 68faca10	FFMA R107, R40, R39, R51	107		
131	00007fb3 68faca20	FFMA R108, R41, R39, R52	108	75	34

SOFTWARE PIPELINE

Prefetch data from shared memory into registers



AVOID STALL SHORT SCOREBOARD

Prefetch data from shared memory into registers

- ▶ Key idea: Overlap LDS and FMA
- ▶ Restructure kernel to prefetch LDS

```
for (int kblock = 0; kblock < K; kblock += KblockSize ) {  
    LDG block[k+1]  
    __syncthreads();  
  
    // Load block[k] from smem to reg  
    // Ctile += Atile x Btile;  
  
    // LDS and FMA block[k]  
    for (int i = 0; i < kblock; i++) {  
        LDS[i+1]  
        FMA[i]  
    }  
  
    STS block[k+1]  
}
```



```
for (int kblock = 0; kblock < K; kblock += KblockSize ) {  
    LDG block[k+1]  
    __syncthreads();  
  
    // Load block[k] from smem to reg  
    // Ctile += Atile x Btile;  
  
    // LDS and FMA block[k]  
    for (int i = 0; i < kblock; i++) {  
        FMA[i]  
        LDS[i+1]  
    }  
  
    STS block[k+1]  
    LDS[i=0]  
}
```

} Prefetch LDS

AVOID STALL SHORT SCOREBOARD

Performance Analysis

- ▶ Current perf
 - ▶ 9769.85 GFlop/s, Time= 7.034 msec
- ▶ Speedup
 - ▶ $9769.85 / 9705.09 = 1.007x$
- ▶ Observation
 - ▶ Stall Short Scoreboard is reduced (0.07->0)
 - ▶ SOL SM is a little increased to 92.77%

Stall Short Scoreboard					
		Current	Baseline 4		
Stall Drain	Stall Not Selected	2.48	2.48	-0.13%	
	Stall Math Pipe Throttle	2.46	2.43	0.98%	
Stall IMC Miss	Selected	1.00	1.00	-0.00%	
	Stall Wait	0.90	0.92	-1.92%	
Stall Misc	Stall Barrier	0.20	0.18	9.33%	
	Stall No Instruction	0.02	0.01	21.40%	
Stall Membar	Stall MIO Throttle	0.01	0.02	-35.86%	
	Stall Branch Resolving	0.01	0.00	inf%	
Stall Sleeping	Stall LG Throttle	0.01	0.00	67.97%	
	Stall Dispatch Stall	0.01	0.02	-73.94%	
Stall Tex Throttle	Stall Long Scoreboard	0.01	0.00	3.78%	
	Stall Short Scoreboard	0.00	0.07	-94.15%	
0.0		Stall Drain	0.00	0.00	12.48%
		Stall IMC Miss	0.00	0.00	-5.55%

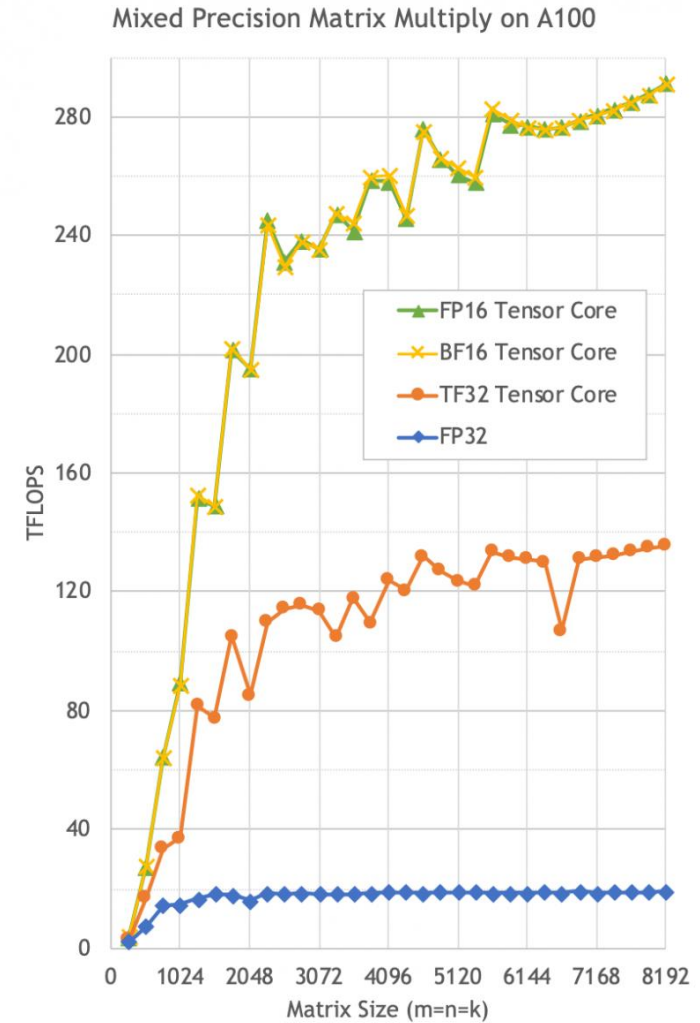
GEMM BASELINES

- ▶ Now we have achieved 9769.85 GFlop/s.
- ▶ According to the specifications of A100 40GB PCIe, FP32 can reach 19.5 TFlop/s. However, BFS A100 is fixed at mclk=1215, pclk=765. So the real SOL GFlop/s should be
 - ▶ $64 \text{ cores/SM} * 108 \text{ SM} * 0.765 \text{ GHz} * 2 \text{ op/FMA} = 10575.36 \text{ GFlop/s}$
- ▶ It means we achieve $9769.85 / 10575.36 = 92.38\%$ SOL.
- ▶ What is the peak GFlop/s we can achieve? Let's try cuBLAS and CUTLASS for comparison.

CUBLAS GEMM

Introduction

- ▶ The cuBLAS Library provides a GPU-accelerated implementation of the basic linear algebra subroutines (BLAS).
- ▶ cuBLAS library uses column-major storage



CUTLASS GTC 2018

CUBLAS GEMM

Gemm Kernel Choice

▶ CUDA 11.1

- ▶ m=n=4096, k=256, 512, 1024
 - ▶ ampere_sgemm_128x64_nn
- ▶ m=n=4096, k=2048, 4096
 - ▶ ampere_sgemm_128x128_nn
- ▶ m=n=4096, k=8192
 - ▶ ampere_sgemm_128x32_sliced1x4_nn

▶ CUDA 11.3

- ▶ m=n=4096, k=256, 512, 1024, 2048, 4096
 - ▶ ampere_sgemm_128x64_nn
- ▶ k=8192
 - ▶ cutlass::Kernel<cutlass_80_simt_sgemm_128x64_8x5_nn_align1>

CUBLAS GEMM

Gemm Perf

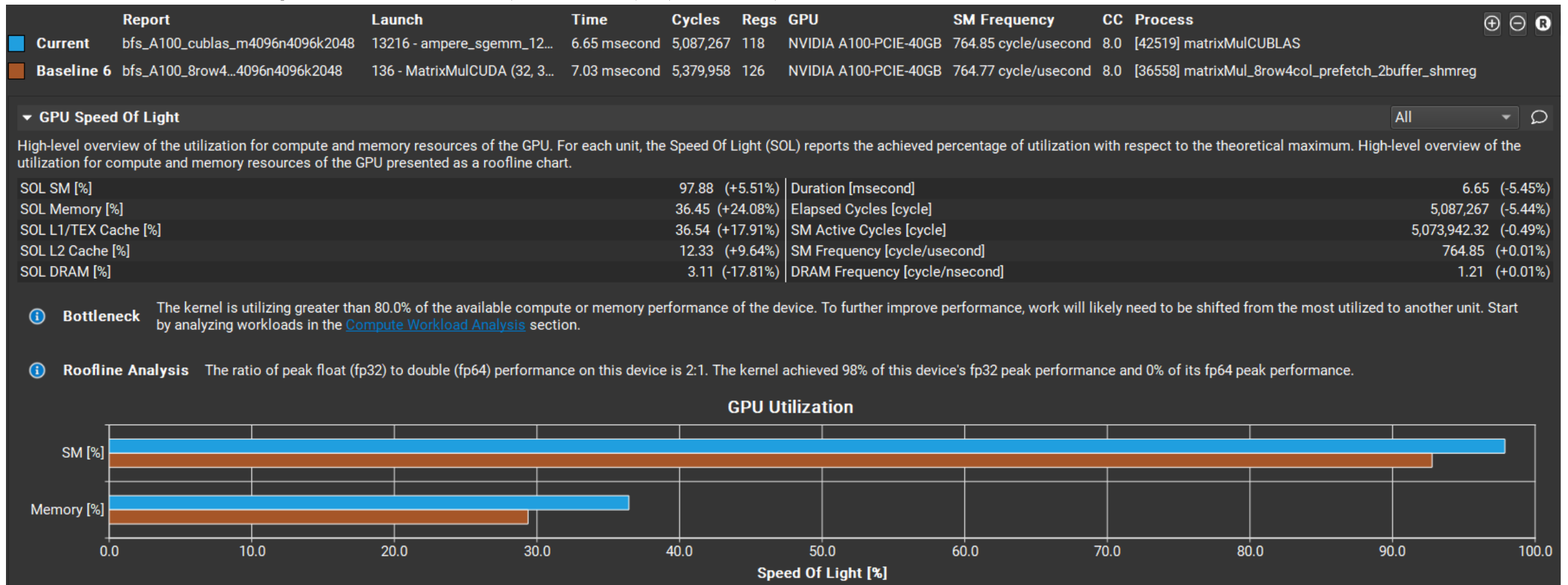
- Fix $M = N = 4096$, fix the $mclk=1215$, $pclk=765$

	CUDA 11.1			CUDA 11.3		
K	Tile size	Time (ms)	GFlop/s	Tile size	Time (ms)	GFlop/s
256	128*64	0.850	10105.70	128*64	0.850	10104.06
512	128*64	1.679	10229.74	128*64	1.677	10241.93
1024	128*64	3.335	10301.72	128*64	3.333	10309.73
2048	128x128	6.655	10326.46	128*64	6.644	10343.19
4096	128x128	13.250	10372.94	128*64	13.269	10357.51
8192	128x32	26.649	10314.76	128*64	26.427	10401.28

CUBLAS GEMM

Gemm Perf

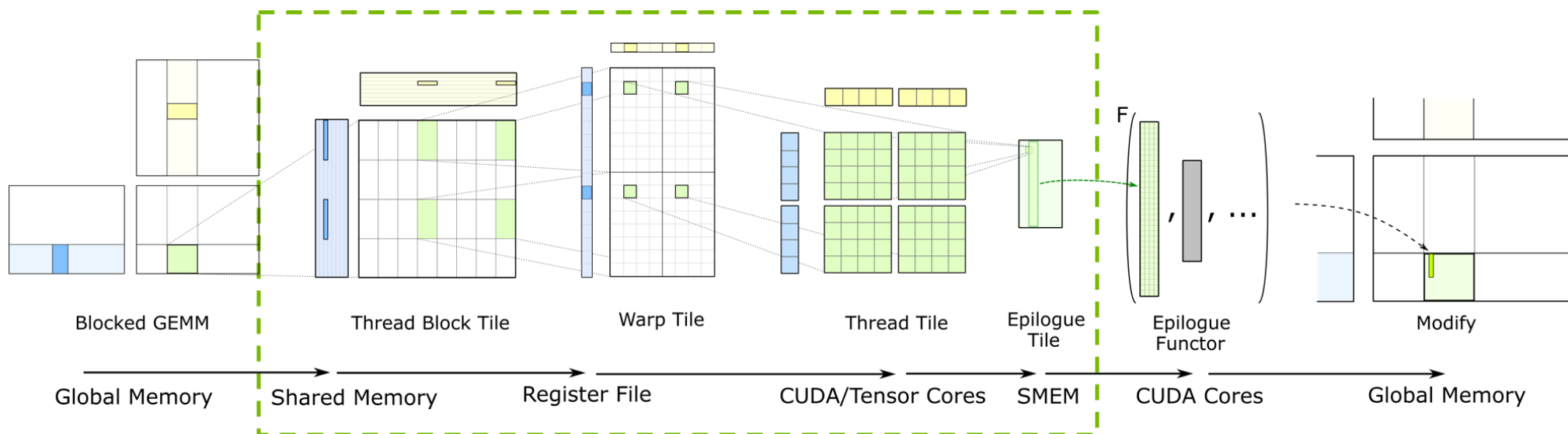
- ▶ Current: cuBLAS (32,32,2)*(256,1,1)
- ▶ Baseline: our implementation (32,32,1)*(32,8,1)



CUTLASS GEMM

Introduction

- ▶ CUDA C++ templates for composable algorithms
- ▶ Performance: **Implement high-performance matrix-multiplication (GEMM)**
- ▶ Structured, reusable components: **flexibility and productivity**



CUTLASS GTC 2018

CUTLASS SGEMM

Try cutlass_profiler command line

- ▶ `./tools/profiler/cutlass_profiler --operation=Gemm --kernels=sgemm --m=4096 --n=4096 --k=2048 --cta_m=128 --cta_n=128 --cta_k=8 --stages=2 --split_k_slices=1 --A=f32:column --min_cc=80 --max_cc=80`
 - ▶ Kernel: `cutlass_simt_sgemm_128x128_8x2_nt_align1`
 - ▶ Arguments: `--A=f32:column --B=f32:row --C=f32:column --alpha=1 --beta=0 --split_k_slices=1 --batch_count=1 --op_class=simt --accum=f32 --cta_m=128 --cta_n=128 --cta_k=8 --stages=2 --warps_m=4 --warps_n=2 --warps_k=1 --inst_m=1 --inst_n=1 --inst_k=1`
 - ▶ Perf: 8915.39 GFlop/s
- ▶ **Issue1**: cannot find kernel of `--A=f32:column --B=f32:row --C=f32:row`
- ▶ **Issue2**: cannot find kernel of `--split_k_slices=2`

CUTLASS SGEMM

Build cutlass sample

- ▶ Based on examples/08_turing_tensorop_gemm/turing_tensorop_gemm.cu
- ▶ Kernel: `_ZN7cutlass6KernelINS_4gemm6ke_1...`
- ▶ Set arguments: `--A=f32:column --B=f32:row --C=f32:row --alpha=1 --beta=0 --split_k_slices=1 --batch_count=1 --op_class=simt --accum=f32 --cta_m=128 --cta_n=128 --cta_k=8 --stages=3 --warps_m=4 --warps_n=2 --warps_k=1 --inst_m=1 --inst_n=1 --inst_k=1 --min_cc=80 --max_cc=80`
- ▶ Perf: **9549.68** GFlop/s
- ▶ **Issue 1**: cannot find kernel of `--stages=2` (error: more than one partial specialization matches the template argument list of class)
- ▶ **Issue 2**: modifications needed to run `--split_k_slices=2`

CUTLASS SGEMM

Try different split_k_slices

- Fix $M = N = 4096$, $K=2048$, fix the mclk=1215, pclk=765

Split-K	Size (Ops)	Time (ms)	GFlop/s
1	68719476736	7.196	9549.68
2	68719476736	6.885	9981.04
4	68719476736	6.951	9886.27
8	68719476736	7.084	9700.66

SPLIT-K OPTIMIZATION

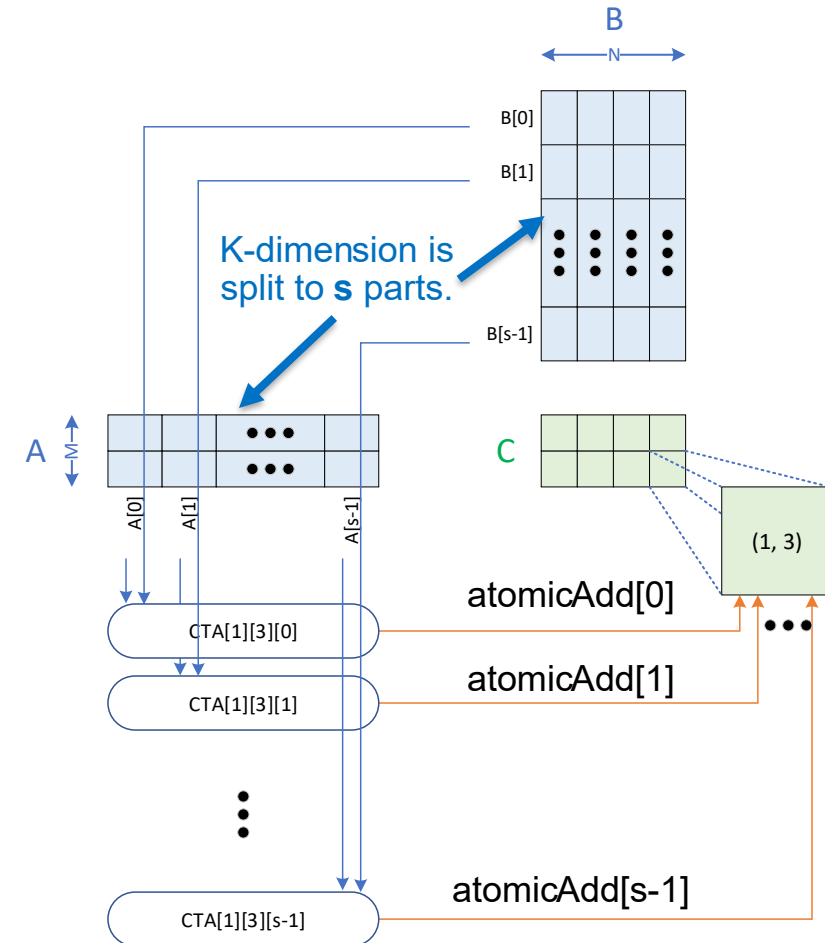
Inter-CTA Split-K reduction

- ▶ In GEMM with small M and N dimension, we may not have enough number of CTAs to saturate all SMs from large GPUs. For example, A100 has 108 SMs and we expect 2 blocks/SM for better perf, which means we need at least 216 blocks.
- ▶ To overcome this, we can split GEMM's K dimension to have more CTAs for better GPU occupancy. Now, for the certain tile of the result matrix, there are multiple CTAs holding partial sum of the final result. Additional reduction is required in GEMM's epilog.
- ▶ Another possible solution is to use smaller tile size like 128*64 rather than 128*128. (not covered here)

SPLIT-K OPTIMIZATION

Atomic reduction

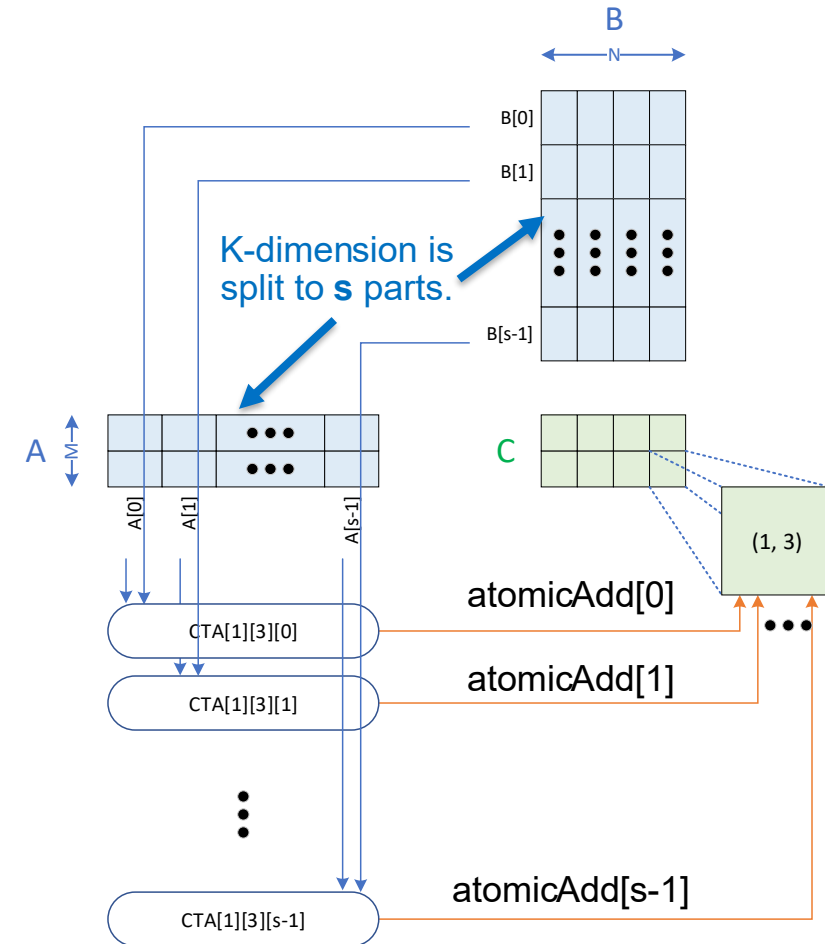
- ▶ In **epilog**, each CTA atomicAdd its temporary data directly to C matrix data in memory.
- ▶ No spinlock read and write are needed. No LDG or STG as well. Use 32-bit red.add instead
 - ▶ atomicAdd doesn't support float4
 - ▶ 1 STG.128 changes to 4*split_k_slices red.add.32
 - ▶ Non-deterministic
 - ▶ Memset(C, 0) is added before Gemm kernel to keep results correct.



ATOMIC REDUCTION

Basic implementation

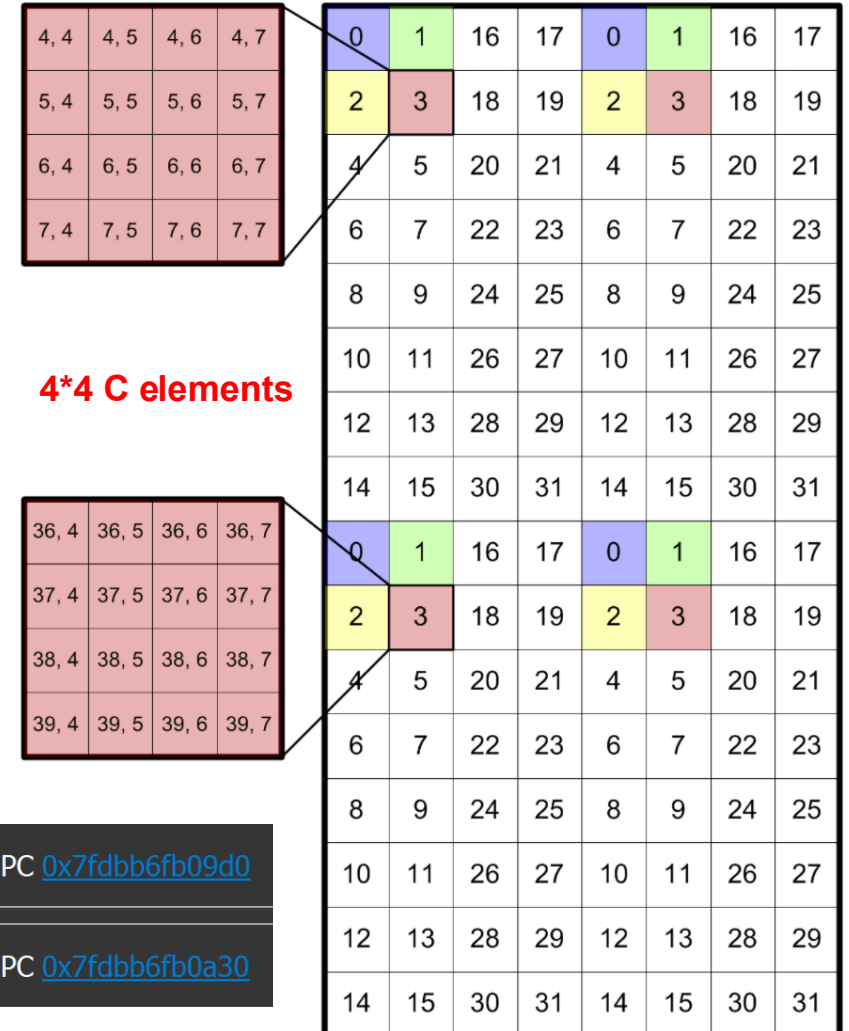
- ▶ Use `grid(split_k_slices* N/128, M/128)` to replace original (N/128, M/128)
- ▶ Each block processes the corresponding slice
- ▶ In epilog, use `atomicAdd` to replace original STG



ATOMIC REDUCTION

Basic implementation: perf analysis

- ▶ Perf: 9674.27 GFlop/s
- ▶ Slower than previous 9769.85 GFlop/s (without split-k)
- ▶ The problem is **uncoalesced global access**, expected 65536 sectors, got 262144 (4.00x)
- ▶ The reason is that each thread processes 4 separated 4*4 contiguous submatrix of C in epilog. But atomidAdd doesn't support float4



⚠ **Uncoalesced Global Accesses** [Warning] Uncoalesced global access, expected 65536 sectors, got 262144 (4.00x) at PC [0x7fdbb6fb09d0](#)

⚠ **Uncoalesced Global Accesses** [Warning] Uncoalesced global access, expected 65536 sectors, got 262144 (4.00x) at PC [0x7fdbb6fb0a30](#)

Figure 4a: A Maxwell SGEMM thread owns 8x8 C elements in 4 4x4 contiguous blocks, a warp owns 64x32 contiguous C elements

ATOMIC REDUCTION

Opt 1: use shared memory to avoid uncoalesced global access

- ▶ Use shared memory of $Mblocksize * NblockSize$
- ▶ The element order in the shared memory is the same with C (contiguous $Mblocksize * NblockSize$)
- ▶ In epilog, `atomicAdd` -> `STS-LDS-atomicAdd`
- ▶ Use the same CTA tile (2row*4column warps) and thread tile (8row*4column threads) for epilog STS.128 with main-loop LDS.128 (swizzled thread id)
- ▶ Use CTA tile (8row*1column warps) and thread tile (1row*32column threads) for epilog LDS.32 and `atomicAdd.32`
- ▶ shared memory size is $128 * 128 * 4 = 64KB$ so we need to use dynamic share memory and set `cudaFuncAttributeMaxDynamicSharedMemorySize`.

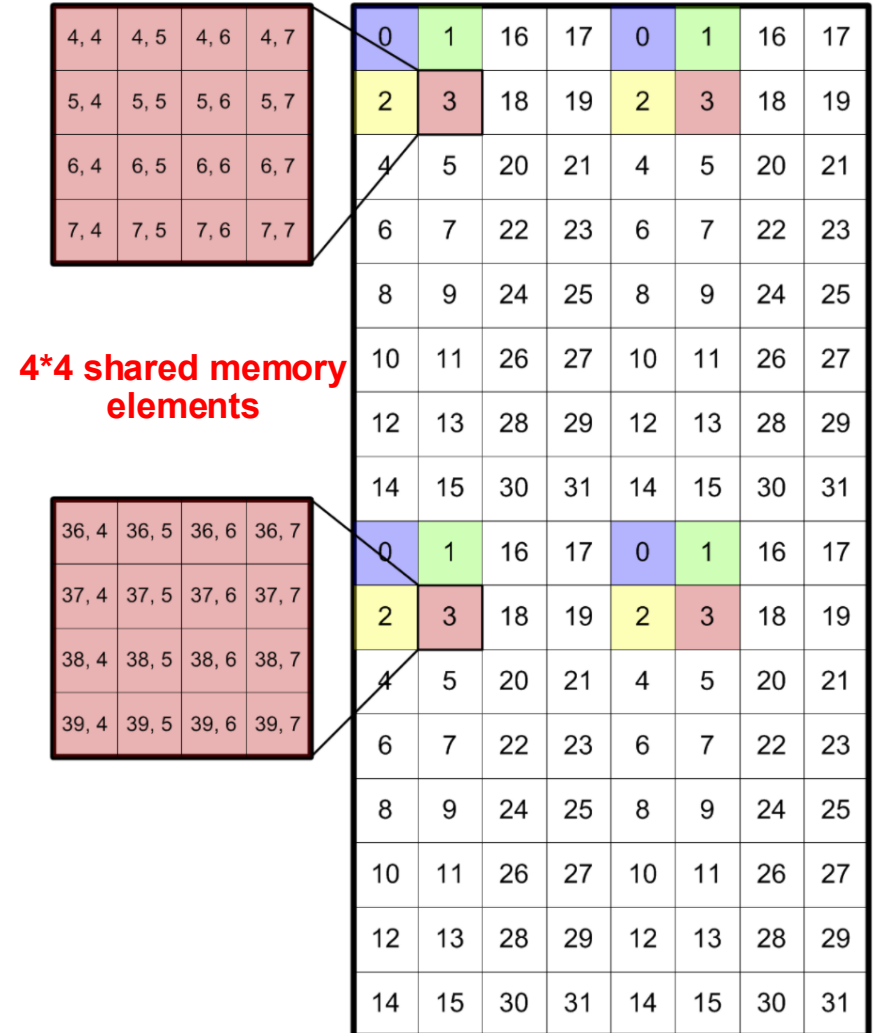


Figure 4a: A Maxwell SGEMM thread owns 8x8 C elements in 4 4x4 contiguous blocks, a warp owns 64x32 contiguous C elements

ATOMIC REDUCTION

Opt 1: perf analysis

- ▶ Perf: **10041.07** GFlop/s
- ▶ 3.8% faster than 9674.27 Gflop/s (basic impl), 2.8% faster than 9769.85 GFlop/s (without split-k)
- ▶ One problem is **uncoalesced shared access**, expected 65536 sectors, got 262144 (4.00x)
- ▶ The reason is that each thread processes 4 separated 4*4 contiguous submatrix of C in epilog. But different threads of a warp use the same bank for STS.128

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	68,157,440	68,157,440	135,600,092	24.19	326,784
Shared Load Matrix	0	0	0	0	0
Shared Store	4,456,448	4,456,448			3,165,346
Shared Store From Global Load	0	0	0	0	5,216
Shared Atomic	0	0	0	0	0
Other	-	-	21,293,458	3.80	0
Total	72,613,888	72,613,888	156,893,550	27.99	3,497,346

ATOMIC REDUCTION

Opt 2: use padding to avoid uncoalesced shared access

- ▶ Try to avoid uncoalesced shared access through padding
- ▶ Shared memory becomes $Mblocksize * (Nblocksize + Paddingsize) = 128 * 132 * 4 = 66KB$, which is too large for V100(96KB), T4(64KB) to support 2 CTAs/SM
- ▶ The better solution is to reuse share memory of main loop $(Kblocksize * (Mblocksize + Nblocksize) * 2) = 16KB$ and use swizzling to avoid uncoalesced shared access in epilog STS.128

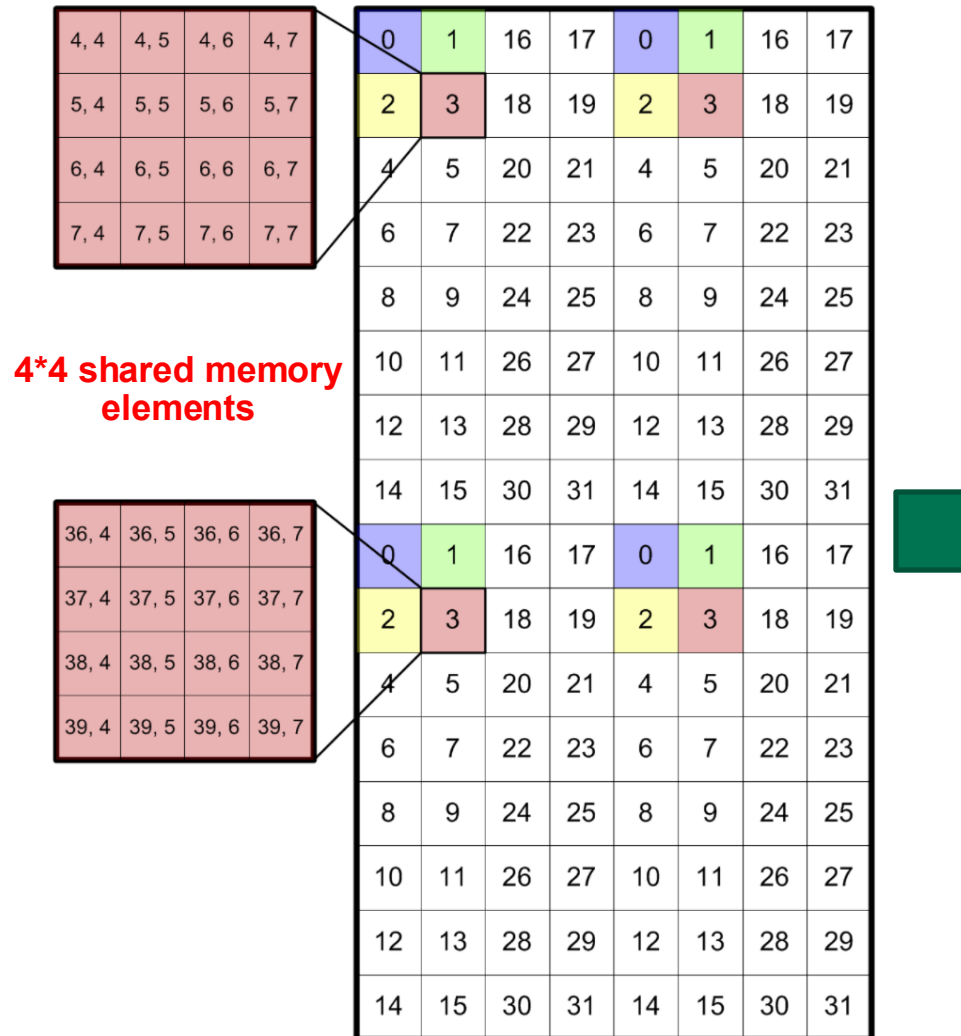


Figure 4a: A Maxwell SGEMM thread owns 8x8 C elements in 4 4x4 contiguous blocks, a warp owns 64x32 contiguous C elements

ATOMIC REDUCTION

Opt 2: perf analysis

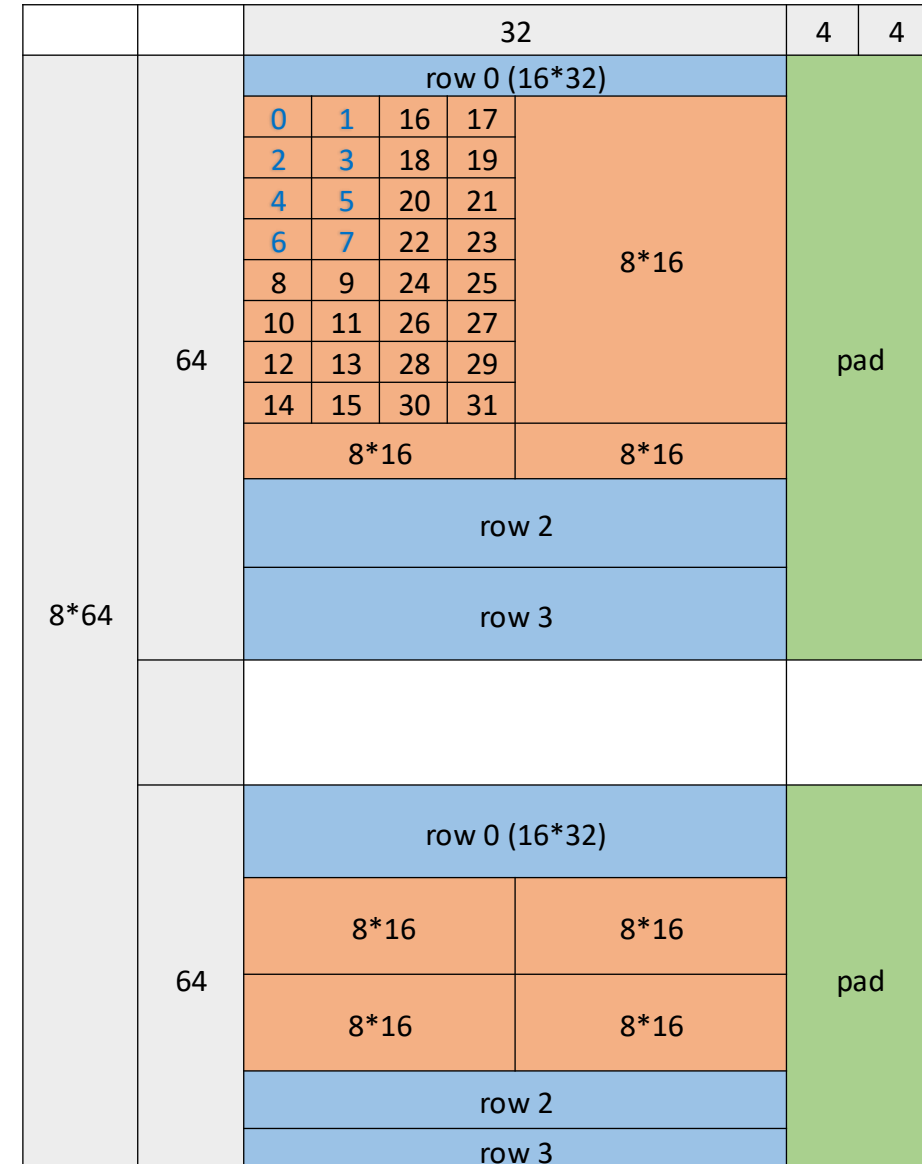
- ▶ Perf: **10074.91** GFlop/s, a little improved
- ▶ **Uncoalesced shared access** still exists, expected 65536 sectors, got 131072 (2.00x)
- ▶ The reason is that each thread processes 4 separated 4*4 contiguous submatrix of C in STS.128. Padding 4 columns cannot scatter threads to different banks

Shared Memory						
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts	
Shared Load	68,157,440 (+0.00%)	68,157,440 (+0.00%)	135,579,923 (-0.01%)	24.27 (+0.34%)	306,159	(-6.31%)
Shared Load Matrix	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0	(+0.00%)
Shared Store	4,456,448 (+0.00%)	4,456,448 (+0.00%)			1,068,365	(-66.25%)
Shared Store From Global Load	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	-272	(-105.21%)
Shared Atomic	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0	(+0.00%)
Other	-	-	19,190,969 (-9.87%)	3.44 (-9.55%)	0	(+0.00%)
Total	72,613,888 (+0.00%)	72,613,888 (+0.00%)	154,770,892 (-1.35%)	27.71 (-1.00%)	1,374,252	(-60.71%)

ATOMIC REDUCTION

Opt 3: reorganize shared memory

- ▶ Design shared memory shape as $8 * (64 * (32 + 4 * 2))$, where 8 is the warp number, $64 * 32$ is the continuous C element sub-block per warp, $4 * 2$ is the padding columns to avoid epilog STS bank conflict.
- ▶ For epilog STS, thread block is still swizzled, $2 * 4$ warp tile, $8 * 4$ thread tile, which is the same as the main loop (LDS and FMA).
- ▶ For epilog LDS and atomicAdd, thread block is original, $8 * 1$ warp tile, $1 * 32$ thread tile, which is the same as prolog (LDG and STS).
- ▶ Now we need 80KB!!! Only A100 (164KB) can support 2 CTAs/SM



ATOMIC REDUCTION

Opt 3: perf analysis

- ▶ Perf: **10101.75** GFlop/s, only a little improved because epilog takes a very small proportion of the whole time
- ▶ **Uncoalesced shared access is removed.**

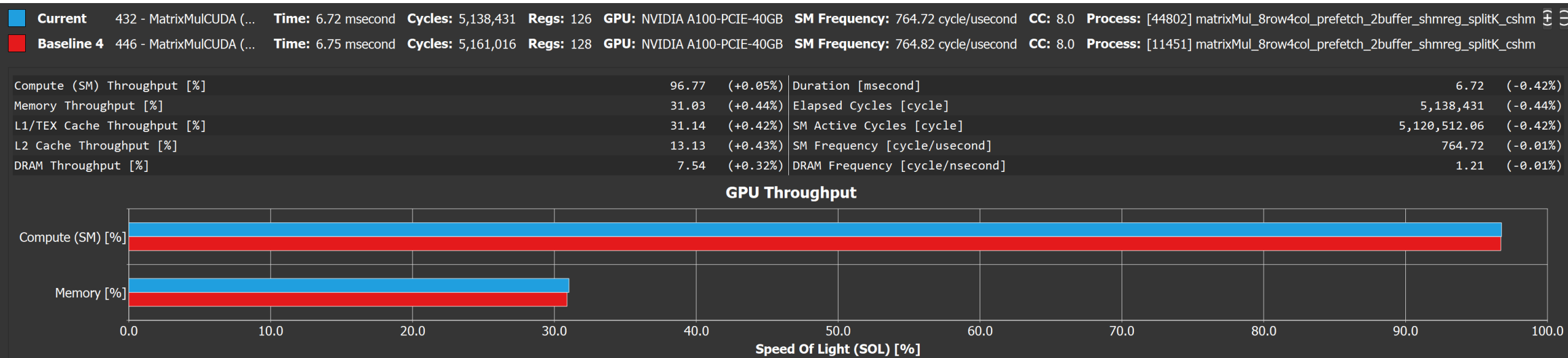
	Report	Launch	Time	Cycles	Regs	GPU	SM Frequency	CC	Process
Current	bfs_A100_8row4col_...e3_m4096n4096k2048	446 - ...	6.75 msecond	5,161,016	128	NVIDIA A100-PCIE-40GB	764.82 cycle/usecond	8.0	[11451] matrixMul_8row4col_prefetch_2buffer_shmreg_splitK_cshm
Baseline 3	bfs_A100_8row4col_...e2_m4096n4096k2048	446 - ...	6.76 msecond	5,171,939	127	NVIDIA A100-PCIE-40GB	764.74 cycle/usecond	8.0	[29863] matrixMul_8row4col_prefetch_2buffer_shmreg_splitK_cshm

Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	68,157,440 (+0.00%)	68,157,440 (+0.00%)	135,606,302 (+0.02%)	24.33 (+0.23%)	327,193 (+6.87%)
Shared Load Matrix	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)
Shared Store	4,456,448 (+0.00%)	4,456,448 (+0.00%)			11,159 (-98.96%)
Shared Store From Global Load	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	-154 (-43.38%)
Shared Atomic	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)	0 (+0.00%)
Other	-	-	18,133,714 (-5.51%)	3.25 (-5.31%)	0 (+0.00%)
Total	72,613,888 (+0.00%)	72,613,888 (+0.00%)	153,740,016 (-0.67%)	27.58 (-0.46%)	338,198 (-75.39%)

ATOMIC REDUCTION

Opt 4: change the writing style of loop

- ▶ It's not an optimization actually. I just change the loop writing style:
 - ▶ Previous: `for (int gBlock = kBlock; gBlock < K / K_UNROLLING; gBlock += kBlock)`
 - ▶ Now: `for (int gBlock = kBlock; gBlock * K_UNROLLING < K; gBlock += kBlock)`
- ▶ Perf: **10143.76 GFlop/s**, a little improved over previous **10101.75 GFlop/s**



ATOMIC REDUCTION

Summary

- ▶ The perf of split-k is much improved but I don't know the reason. Maybe main loop and prolog and epilog have a better overlap after blocks become more.
- ▶ Memset (Matrix C: $4096 \times 4096 \times 4 = 64\text{MB}$) takes 0.055 ms (1163.63 GB/s, compared with memory bandwidth 1555 GB/s)

Baseline	Split-K	Time (ms)	GFlops/s	SOL
matrixMul	None	7.034	9769.94	92.38%
cutlass	2	6.885	9981.04	94.38%
matrixMul_splitK	2	6.775	10143.76	95.92%
cublas	2	6.655	10326.46	97.65%

ATOMIC REDUCTION

What's the impact of split-k?

- Fix $M = N = 4096$, fix the $mclk=1215$, $pclk=765$
- Split-k is better with larger K, when $M = N = 4096$

	Not split-k		Split-k		Speed-up
K	Time (ms)	GFlop/s	Time (ms)	GFlop/s	Split-k/Not
256	0.909	9452.73	0.968	8877.19	0.94
512	1.782	9639.16	1.796	9567.86	0.99
1024	3.532	9727.41	3.454	9946.79	1.02
2048	7.034	9770.00	6.776	10141.93	1.04
4096	14.032	9794.77	13.411	10248.11	1.05
8192	28.030	9806.53	26.694	10297.43	1.05

ATOMIC REDUCTION

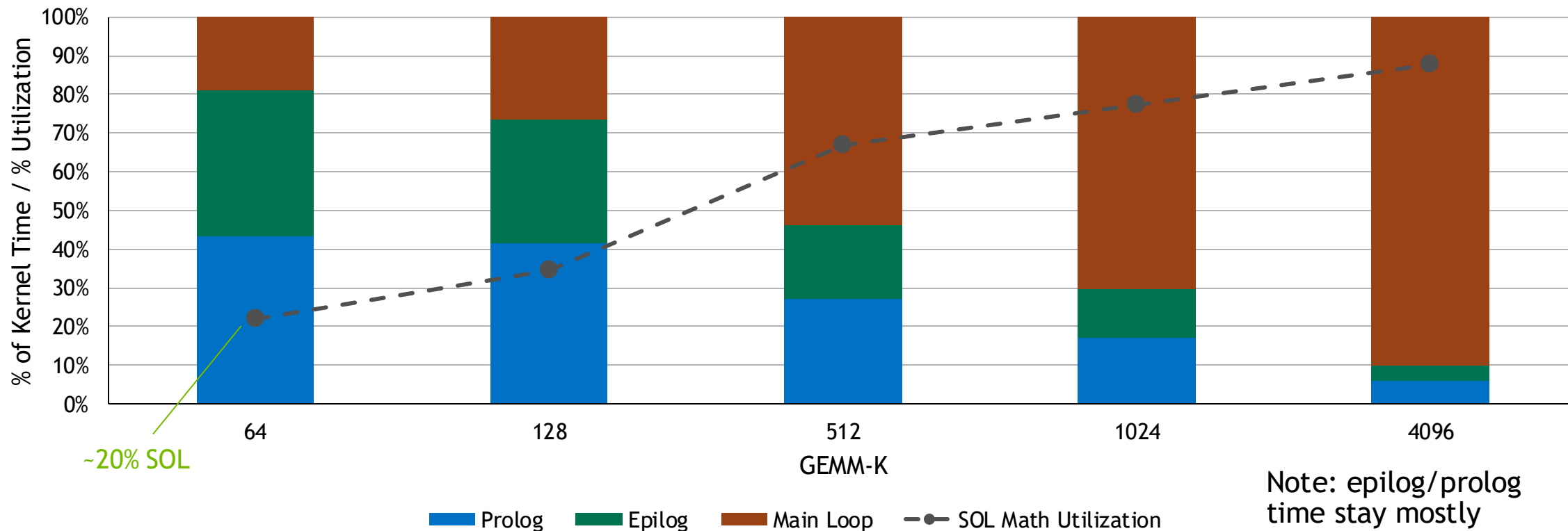
What's the impact of split-k?

- Fix K=2048, fix the mclk=1215, pclk=765, use “not split-k” perf as base
- The perf depends on the problem size (M, N, K)

		Not split-k	Split-k		cuBLAS	
M	N	GFlop/s	GFlop/s	Speed-up	GFlop/s	Speed-up
1024	1024	5950.73	5885.71	0.99	9280.29	1.56
2048	2048	8084.29	9528.41	1.18	9730.67	1.20
4096	4096	9727.41	10142.95	1.04	10326.32	1.06
3072	1152	10169.48	10002.03	0.98	10117.65	0.99
3072	1152*2	10247.63	10095.87	0.99	10365.79	1.01
3072*2	1152*4	10315.35	10185.73	0.99	10403.07	1.01

RETHINK GEMM

What's the impact of K



~20% SOL

Low GEMM-K → High prolog/epilog overhead + lower utilization

Note: epilog/prolog time stay mostly constant across different values of K

RETHINK GEMM

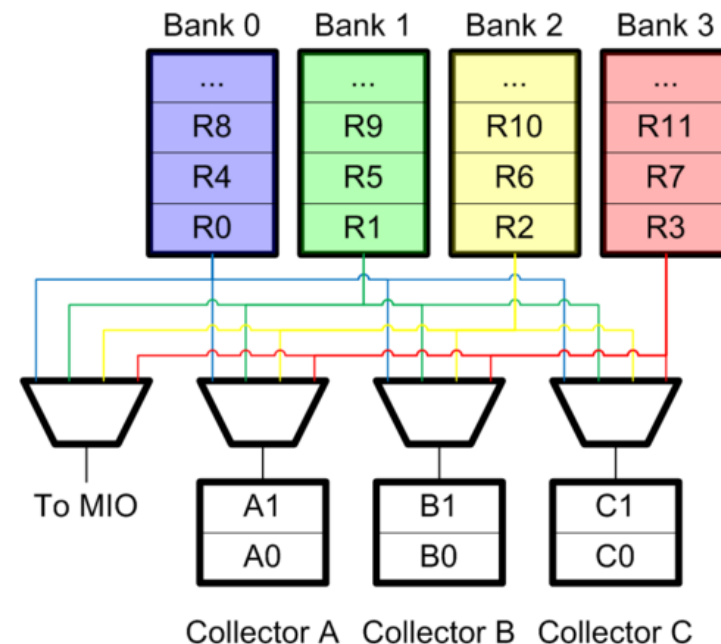
Why is our kernel still slower than cuBLAS?

- ▶ Register control
 - ▶ Reuse
 - ▶ Bank conflict
- ▶ L2 cache hit rate

MAXIMIZING RF BANDWIDTH

Take advantage of reuse yet again

- ▶ FFMA requires three values
 - ▶ Must complete in 2 cycles to avoid stall
- ▶ Operand collectors save us time and energy
 - ▶ Operand collectors hold up to two values
 - ▶ Compiler marks operand with .reuse or omits on last use
 - ▶ Collectors invalidated when warp goes off deck



https://wiki.nvidia.com/gpuhwkepler/index.php/Compute/Application_Performance/Applications/Linear_Algebra/GEMM/GEMM_for_Dummies

RETHINK GEMM

Why is our kernel still slower than cuBLAS?

- ▶ Reuse flag
- ▶ Use internal nsight compute package
- ▶ Our implementation has many “WAIT2_END_GROUP”

Current

Launch

Time

Cycles

Regs

GPU

SM Frequency

432 - MatrixMulCUDA (64, 32, 1)x(32, 8, 1)

6.75 msecond

5,158,223

128

NVIDIA A100-PCIE-40GB

764.45 cycle/usecond

View: SASS

Source: MatrixMulCUDA

reuse

Navigation: Instructions Executed

#	Address	Source	Annotation	Pipeline	Live Registers	np_sample_count
241	00007f82 f2face00	FFMA R85, R32, R48, R85	?WAIT2_END_GROUP	FMA	121	293
242	00007f82 f2face10	FFMA R83, R33, R48.reuse, R83	?WAIT2	FMA	121	4,294
243	00007f82 f2face20	FFMA R57, R34, R48.reuse, R55	?WAIT2	FMA	121	257
244	00007f82 f2face30	FFMA R96, R35, R48, R41	?WAIT2	FMA	121	264
245	00007f82 f2face40	FFMA R97, R36, R49.reuse, R97	?WAIT2	FMA	121	252
246	00007f82 f2face50	FFMA R90, R37, R49.reuse, R90	?WAIT2	FMA	121	264
247	00007f82 f2face60	FFMA R91, R38, R49, R91	?WAIT2_END_GROUP	FMA	121	268
248	00007f82 f2face70	FFMA R92, R39, R49.reuse, R92	?WAIT2	FMA	121	3,379
249	00007f82 f2face80	FFMA R93, R32, R49.reuse, R93	?WAIT2	FMA	121	253
250	00007f82 f2face90	FFMA R94, R33, R49.reuse, R94	?WAIT2	FMA	121	283
251	00007f82 f2facea0	FFMA R95, R34, R49.reuse, R95	?WAIT2	FMA	121	253
252	00007f82 f2faceb0	FFMA R102, R35, R49, R102	?WAIT2	FMA	121	257
253	00007f82 f2facec0	FFMA R105, R36, R50, R105	?WAIT2_END_GROUP	FMA	121	280
254	00007f82 f2faced0	FFMA R104, R37, R50.reuse, R104	?WAIT2	FMA	121	2,579
255	00007f82 f2facee0	FFMA R99, R38, R50.reuse, R99	?WAIT2	FMA	121	292
256	00007f82 f2facef0	FFMA R98, R39, R50.reuse, R98	?WAIT2	FMA	121	265
257	00007f82 f2facf00	FFMA R101, R32, R50.reuse, R101	?WAIT2	FMA	121	247
258	00007f82 f2facf10	FFMA R100, R33, R50.reuse, R100	?WAIT2	FMA	121	244
259	00007f82 f2facf20	FFMA R103, R34, R50, R103	?WAIT2_END_GROUP	FMA	121	270

RETHINK GEMM

Why is our kernel still slower than cuBLAS?

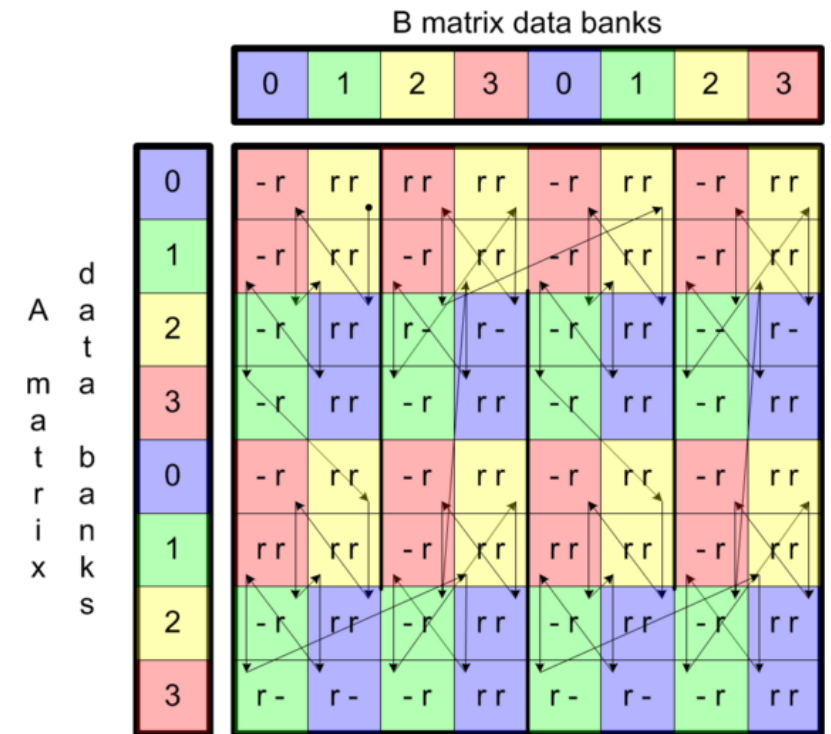
- cuBLAS kernel has more .reuse
- Only WAIT2

Launch		Time	Cycles	Regs	GPU	SM Frequency
13058 - ampere_sgemm_128x128_nn (32, 32, 2)x(256, 1, 1)		6.65 msecond	5,087,698	118	NVIDIA A100-PCIE-40GB	764.81 cycle/usecond
View: SASS						
Source: ampere_sgemm_128x128_nn		reuse	Navigation: Instructions Executed			
#	Address	Source	Annotation	Pipelines	Live Registers	np_sample_count
494	00007f53 df8437d0	FFMA R24, R87, R101.reuse, R24	?WAIT2	FMA	115	166
495	00007f53 df8437e0	FFMA R33, R88.reuse, R100.reuse, R33	?WAIT2	FMA	115	316
496	00007f53 df8437f0	FFMA R32, R88, R101.reuse, R32	?WAIT2	FMA	115	298
497	00007f53 df843800	FFMA R41, R89.reuse, R100.reuse, R41	?WAIT2	FMA	115	261
498	00007f53 df843810	FFMA R40, R89, R101.reuse, R40	?WAIT2	FMA	115	290
499	00007f53 df843820	FFMA R49, R90.reuse, R100.reuse, R49	?WAIT2	FMA	115	259
500	00007f53 df843830	FFMA R48, R90.reuse, R101.reuse, R48	?WAIT2	FMA	115	343
501	00007f53 df843840	FFMA R57, R91.reuse, R100, R57	?WAIT2	FMA	115	292
502	00007f53 df843850	FFMA R56, R91.reuse, R101, R56	?WAIT2	FMA	115	327
503	00007f53 df843860	FFMA R59, R91.reuse, R102.reuse, R59	?WAIT2	FMA	115	260
504	00007f53 df843870	FFMA R58, R91, R103.reuse, R58	?WAIT2	FMA	115	291
505	00007f53 df843880	FFMA R51, R90.reuse, R102.reuse, R51	?WAIT2	FMA	115	291
506	00007f53 df843890	FFMA R50, R90, R103.reuse, R50	?WAIT2	FMA	115	258
507	00007f53 df8438a0	FFMA R43, R89.reuse, R102.reuse, R43	?WAIT2	FMA	115	272
508	00007f53 df8438b0	FFMA R42, R89, R103.reuse, R42	?WAIT2	FMA	115	291
509	00007f53 df8438c0	FFMA R35, R88.reuse, R102.reuse, R35	?WAIT2	FMA	115	279
510	00007f53 df8438d0	FFMA R34, R88, R103.reuse, R34	?WAIT2	FMA	115	301
511	00007f53 df8438e0	FFMA R27, R87.reuse, R102.reuse, R27	?WAIT2	FMA	115	248
512	00007f53 df8438f0	FFMA R26, R87, R103.reuse, R26	?WAIT2	FMA	115	276

DRUNKARD'S HOPSCOTCH

Cublas SGEMM RF Mapping to Minimize Bank Conflicts

- ▶ Figure at right illustrates CUBLAS SGEMM RF mapping
 - ▶ Colors indicate RF bank assignment
 - ▶ 8x8 tile indicates accumulator bank mapping
 - ▶ Arrows indicate sequence of FFMAs with reuse pattern:
 - ▶ rr .reuse operand A, .reuse operand B
 - ▶ r- .reuse operand A
 - ▶ -r .reuse operand B



https://wiki.nvidia.com/gpuhwkepler/index.php/Compute/Application_Performance/Applications/Linear_Algebra/GEMM/GEMM_for_Dummies

RETHINK GEMM

Why is our kernel still slower than cuBLAS?

- ▶ L2 cache hit rate
- ▶ Remapping CTA ids to increase L2 data sharing and decrease DRAM accesses

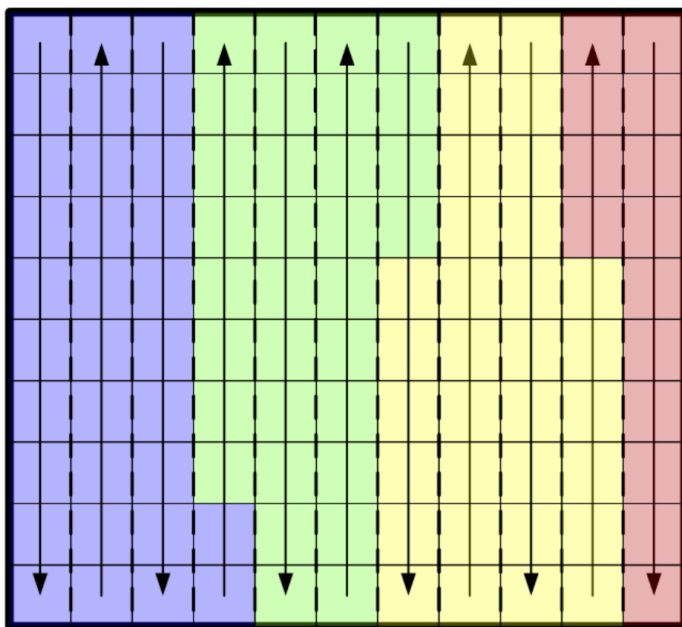


Figure 12a: CTAs per wave = 32,
groupCols = 1, fetched = 54

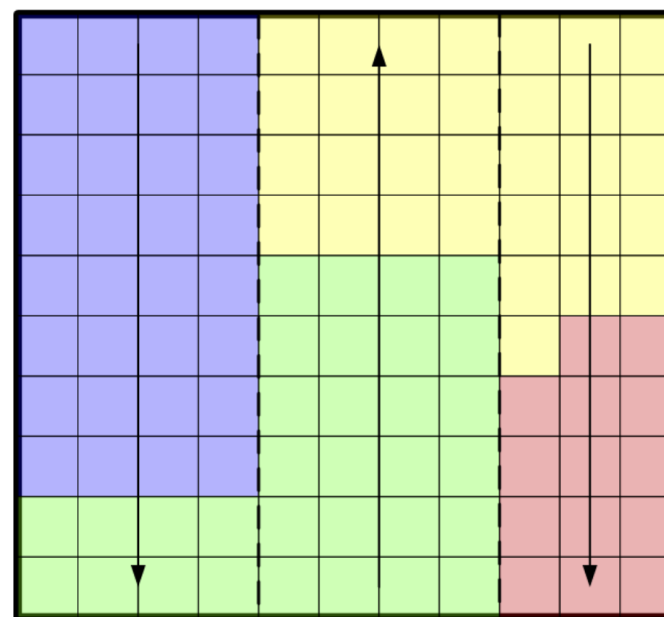


Figure 12c: CTAs per wave = 32,
groupCols = 4, fetched = 47

https://wiki.nvidia.com/gpuhwkepler/index.php/Compute/Application_Performance/Applications/Linear_Algebra/GEMM/GEMM_for_Dummies

RETHINK GEMM

What can we further explore?

- ▶ LDGSTS
- ▶ Multiple-stage like 3-stage
- ▶ Smaller tile size like 128×64
- ▶ Split-k with smaller shared memory size (swizzling, no padding)
- ▶ Split-k with spinlock (serial reduction)
- ▶ ...

GEMM OPTIMIZATION STAGE 1

Final Perf

Step	Perf (GFlop/s)	Percentage of Peak Perf (%)	Time (ms)	Cuda Block Size	Corresponding C Block Size	Thread Tile Size	Improvement Percentage	Memory Techniques
Base Line	1861.22	17.63%	36.922	(32, 32)	(32, 32)	/	0.00%	No shared memory
shared memory	2862.98	27.11%	24.003	(32, 32)	(32, 32)	/	53.82%	A, B shared memory
shared memory + thread tile	7989.69	75.67%	8.601	(32, 32)	(128, 128)	(4, 4)	329.27%	A, B shared memory, Csub on Registers
Thread_Tile_8_4	8431.95	79.86%	8.15	(32, 16)	(128, 128)	(8, 4)	353.03%	A, B shared memory, Csub on Registers
Thread_Tile_16_4	8642.93	81.85%	7.951	(32, 8)	(128, 128)	(16, 4)	364.37%	A, B shared memory, Csub on Registers
Swizzle+Float4	9159.75	86.75%	7.502	(32, 8)	(128, 128)	(8, 8)	392.14%	A, B shared memory, Csub on Registers

GEMM OPTIMIZATION STAGE 2

Final Perf

Step	Perf (GFlop/s)	Percentage of Peak Perf (%)	Time (ms)	Cuda Block Size	Corresponding C Block Size	Thread Tile Size	Improvement Percentage	Memory Techniques
Prefetch LDG	9720.78	91.92%	7.069	(32, 8)	(128, 128)	(8, 8)	422.28%	A, B shared memory, Csub on Registers
Double buffer	9705.09	91.77%	7.081	(32, 8)	(128, 128)	(8, 8)	421.44%	A, B shared memory, Csub on Registers
Prefetch LDS	9769.85	92.38%	7.034	(32, 8)	(128, 128)	(8, 8)	424.91%	A, B shared memory, Csub on Registers
SplitK	9674.27	91.48%	7.103	(32, 8)	(128, 128)	(8, 8)	419.78%	A, B shared memory, Csub on Registers
SplitK+shared memory	10143.76	95.92%	6.775	(32, 8)	(128, 128)	(8, 8)	445.00%	A, B shared memory, Csub on Registers
Cutlass	9981.04	94.38%	6.885	(32, 8)	(128, 128)	(8, 8)	436.26%	A, B shared memory, Csub on Registers
cuBLAS	10326.46	97.65%	6.655	(32, 8)	(128, 128)	(8, 8)	454.82%	A, B shared memory, Csub on Registers

SUMMARY

- ▶ Avoid fetching redundant information from global memory (shared memory)
- ▶ Avoid fetching redundant information from shared memory (outer loop)
- ▶ Thread tile is large enough to make FMA the bottleneck (8*8 thread tile)
- ▶ Use LDS.128 for doubling shared memory bandwidth (float4, thread id swizzling)
- ▶ Avoid stalls on shared and global memory loads (LDS prefetching from shared memory, LDG prefetching from global memory, double buffer, register reuse, register bank conflict)
- ▶ Threads use sufficiently few registers (also shared memory) to have enough blocks
- ▶ Of course, avoid uncoalesced global access and bank conflict

QUESTIONS?

- ▶ Code available:
 - ▶ https://gitlab-master.nvidia.com/mengw/cuda_practice/-/tree/main/Gemm
- ▶ Really appreciate for Petrick's great help!